

Dragon: A Fast Hash Family Design

Meiqin Wang^{1,3,5}, Kai Hu^{1,3,4}, Zhuolong Zhang^{1,3}, Yong Fu^{1,3,5}, Yanhong Fan^{1,3}, Lixuan Wu^{1,3}, Shiyao Chen^{1,3}, Jiamin Cui^{1,3}, Zhongfeng Niu², Qun Liu^{1,3}, Chenhao Jia^{1,3} and Longzheng Cui^{1,3}

¹ School of Cyber Science and Technology, Shandong University, Qingdao, China

mqwang@sdu.edu.cn, kai.hu@sdu.edu.cn, zhuolongzhang@mail.sdu.edu.cn,

² School of Physical and Mathematical Sciences, Nanyang Technological University, Singapore

³ State Key Laboratory of Cryptography and Digital Economy Security, Shandong University, Qingdao, China

⁴ Suzhou Research Institute, Shandong University, Suzhou, 215123, China

⁵ Quan Cheng Shandong Laboratory, Jinan, China

1 Introduction

Cryptographic hash functions are a core building block of modern cryptography. They underlie digital signatures, message authentication, key derivation, and commitments. Their strength is measured chiefly by resistance to collision, preimage, and second-preimage attacks. The ongoing migration to post-quantum cryptography raises their importance further. Hash functions are the sole primitive behind hash-based signatures [Mer79, LM95, CAD⁺20], one of the most conservative families of post-quantum signatures. They also remain robust against quantum adversaries on their own: Grover’s algorithm [Gro96] speeds up preimage search only quadratically, and quantum collision search [BHT98] yields a limited advantage, so an h -bit classical security level retains a substantial margin in the quantum setting. High security levels, such as 1024-bit preimage resistance, are therefore attractive as quantum computing advances. Driven by these demands, several bodies have launched efforts to develop the next generation of hash standards. The current Chinese commercial hash standard is SM3 [Sta10]. The Chinese call for next-generation commercial cryptographic algorithms [NIC25] seeks its successors, targeting security levels up to 1024 bits with strong (second) preimage resistance in addition to collision resistance.

Many recent hash designs are permutation-based and follow the sponge construction [BDPVA07, BDPVA11] that the SHA-3 standard [D⁺15] made popular. A sponge with capacity c provides only about $c/2$ bits of (second) preimage resistance [AMP11, LM22]. Reaching h -bit (second) preimage security therefore forces $c \geq 2h$, and SHA-3 follows this rule with $c = 2h$. A 1024-bit (second) preimage target then forces $c = 2048$ and a permutation of width above 2048 bits. Such a permutation is expensive to design and to evaluate, and its cost is paid by every member of the family.

To overcome this, we designed a new mode of operation. Our sponge-forward XOF construction **Sponge-F^{P, pd10*}** [GHJ⁺25], to appear at CRYPTO 2026, reaches high (second) preimage security with a substantially smaller permutation than a standard sponge requires. It feeds the inner part of the state forward at each permutation call. This raises (second) preimage resistance close to the output length h while keeping the capacity at $c \approx h + 64$, instead of $c \geq 2h$. A constant injected in the final block prevents the length-extension attacks that squeezing the inner part would otherwise allow [AMP12, HPY12]. Its collision, preimage, and indistinguishability security are established in the random-permutation model.

We instantiate $\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ into *Dragon*, a family of fixed-length hash functions and arbitrary-length extendable-output functions (XOFs) built on a single 1600-bit permutation, *Dragon-p*. A 1600-bit width then reaches every (second) preimage target up to 1024 bits, where a standard sponge would need a permutation of width above 2048 bits. The members of the *Dragon* hash family are

- Dragon-1024
- Dragon-768
- Dragon-512
- Dragon-XOF-256
- Dragon-XOF-384
- Dragon-XOF-512

differentiated by their claimed security level against preimage attacks. The details of the six members are shown in Table 1. For example, *Dragon*-1024 provides 1024-bit security against a preimage attack.

Table 1: Overview of the members in the *Dragon* family. For the XOF instances, ν denotes the arbitrary output length, and the preimage and collision security are capped at the target level by the capacity.

Name	Size		Security		Data limits (bits)
	State	Hash	Preimage	Collision	
Dragon-1024	1,600	1024	1024	512	2^{64}
Dragon-768	1,600	768	768	384	2^{64}
Dragon-512	1,600	512	512	256	2^{64}
Dragon-XOF-256	1,600	ν	$\min\{256, \nu\}$	$\min\{256, \nu/2\}$	2^{64}
Dragon-XOF-384	1,600	ν	$\min\{384, \nu\}$	$\min\{384, \nu/2\}$	2^{64}
Dragon-XOF-512	1,600	ν	$\min\{512, \nu\}$	$\min\{512, \nu/2\}$	2^{64}

The state transformation shared by all members is a new 1600-bit permutation, *Dragon-p*. It operates on a 5×5 array of 64-bit lanes and applies 16 rounds of ARX-based mixing, in which modular addition is the only nonlinear operation. The permutation is built for high throughput on 64-bit software platforms. Because it uses only universal integer instructions, it needs no dedicated cryptographic instruction-set support (such as Intel’s AES-NI or SHA Extensions, or the Armv8 Cryptographic Extension) and runs at full speed on any platform. On both x86 and ARM, every fixed-output *Dragon* instance hashes faster than SHA3-512 at all tested input lengths. Relative to the current SM3 standard, *Dragon* provides higher security levels at competitive software speed: *Dragon*-512 and *Dragon*-768 are faster than SM3 at every tested input length on both platforms, while *Dragon*-1024 reaches 1024-bit (second) preimage security at a comparable long-message cost.

Contributions. The contributions of this work are as follows.

- We design the *Dragon* hash family. It instantiates our sponge-forward mode $\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ [GHJ+25] over a single 1600-bit permutation, and it reaches up to 1024-bit (second) preimage and 512-bit collision security without the $c \geq 2h$ capacity that a standard sponge requires.

- We design **Dragon- p** , a new ARX permutation of width 1600 bits. Its round function uses modular addition as the only source of nonlinearity, together with word rotations and a linear mixing layer.
- We analyze the security of **Dragon**. The analysis covers differential and linear cryptanalysis, differential-linear cryptanalysis, integral distinguishers, impossible-differential and zero-correlation distinguishers, algebraic attacks, quantum attacks, statistical randomness tests, and length-extension resistance.
- We implement **Dragon** in software and benchmark it against SHA3-512 on x86-64 (Intel Xeon Platinum 8358) and ARM (Rockchip RK3588). All three fixed-output instances are faster than SHA3-512 at every tested message length. On long messages, **Dragon-512**, **Dragon-768**, and **Dragon-1024** reach 4.87, 6.49, and 9.70 cycles per byte on x86-64 and 4.47, 5.96, and 8.99 on ARM, against 12.46 and 15.49 for SHA3-512, while offering higher (second) preimage security.

The remainder of this document is organized as follows. Section 2 specifies the **Dragon** family, including the sponge-forward mode, the **Dragon- p** permutation and hash instances. Section 3 presents the design rationale and justifies the parameter choices to achieve the security targets. Section 4 analyzes the security of the mode, the permutation, and the overall hash family. Section 5 reports the software and hardware performance results. Finally, Section 6 concludes with a summary of the main features of the submission.

2 Specification

2.1 Overview

Dragon is a family of cryptographic hash functions and extendable-output functions (XOFs) built around a public 1600-bit permutation. Its design follows a sponge-forward paradigm, in which a permutation-based state is used to absorb message blocks and generate hash or XOF outputs. The construction consists of two components: the sponge-forward mode, which defines the external hashing interface, and the permutation **Dragon- p** , which provides the underlying state transformation.

The **Dragon- p** operates on a 5×5 array of 64-bit lanes and applies 16 rounds of ARX-based mixing. It serves as the common cryptographic primitive for all members of the family, while the sponge-forward mode specifies the procedures for message absorption, padding, and output extraction.

The **Dragon** family contains three fixed-output variants, **Dragon-512**, **Dragon-768**, and **Dragon-1024**, together with the three extendable-output functions **Dragon-XOF-256**, **Dragon-XOF-384**, and **Dragon-XOF-512**. All variants use the same 1600-bit state and the same 16-round **Dragon- p** . They differ only in their rate r , capacity $c = 1600 - r$, output length, and public initialization vector. Table 2 gives a compact overview of the six variants; the lane-level parameters and IV values are specified later in the hash-computation subsection.

Table 2: Overview of the members of the **Dragon** family.

Variant	Capacity (bits)	Rate (bits)	Output (bits)
Dragon-512	576	1024	512
Dragon-768	832	768	768
Dragon-1024	1088	512	1024
Dragon-XOF-256	576	1024	ν
Dragon-XOF-384	832	768	ν
Dragon-XOF-512	1088	512	ν

2.2 Notation and encoding conventions

The following notation is used throughout the paper.

For a nonnegative integer n , let $\{0, 1\}^n$ denote the set of bit strings of length n , and let $\{0, 1\}^*$ denote the set of all finite bit strings. The length of a bit string X is denoted by $|X|$, and concatenation is denoted by $X\|Y$. For $0 \leq t \leq |X|$, the notation $\text{left}_t(X)$ denotes the leftmost t bits of X , and $\text{right}_t(X)$ denotes the rightmost t bits of X . Bitwise XOR is denoted by $\oplus$.

The ARX operations of the Dragon permutation are performed on 64-bit words, identified with elements of $\mathbb{Z}/2^{64}\mathbb{Z}$. Addition modulo 2^{64} is denoted by $\boxplus$, and $x \lll a$ denotes the cyclic left rotation of x by $a \bmod 64$ bit positions. Throughout the paper, a 64-bit word x is represented as

$$x = \sum_{z=0}^{63} x[z] \cdot 2^z, \quad x[z] \in \{0, 1\},$$

where $x[0]$ and $x[63]$ denote the least and most significant bits of x , respectively.

2.3 The mode: sponge-forward

$\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ is our sponge-forward construction [GHJ⁺25], defined upon a permutation $P : \{0, 1\}^b \mapsto \{0, 1\}^b$ and the injective padding function

$$\text{pd}10^*(M) := M\|10^{r - ((|M|+1) \bmod r)}$$

of [BDPVA11]. Its minimal and maximal output lengths are $\nu_{\min}$ and $\nu_{\max}$ respectively. Let $M \in \{0, 1\}^*$ and ν be an integer such that $\nu_{\min} \leq \nu \leq \nu_{\max}$. On input (M, ν) , $\text{Sponge-F}^{\text{P}, \text{pd}10^*}(M, \nu)$ outputs a string $Z \in \{0, 1\}^\nu$. $\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ has two additional parameters $r, c \in \mathbb{N}$ and a fixed public constant $IV \in \{0, 1\}^c$, where $c \geq \nu_{\min}$, $r + c = b$, where r , resp. c , is the bit length of the “sponge” outer, resp. inner, part of the b -bit internal state. The concrete process of $\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ is described by the algorithms in Fig. 1 and illustrated in Fig. 2.

$\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ can be naturally instantiated as a XOF. For a fixed digest length $h < c$, it instead yields a fixed-output hash. A single squeeze then produces the whole digest, so the squeezing phase makes no additional permutation call, and the digest is read directly from the inner part of the final state. We depict this case in Fig. 6 and write

$$\text{Sponge-F}_h^{\text{P}, \text{pd}10^*}(\cdot) := \text{Sponge-F}^{\text{P}, \text{pd}10^*}(\cdot, h), \quad (1)$$

for the corresponding fixed-output instantiation.

2.4 The permutation: Dragon-p

Dragon employs a 1600-bit ARX permutation

$$P : \{0, 1\}^{1600} \rightarrow \{0, 1\}^{1600}$$

as the underlying primitive of the sponge-forward construction. The permutation operates on a 5×5 array of 64-bit lanes and combines modular addition, cyclic rotation, and bitwise XOR to achieve nonlinear mixing and diffusion. The following subsections describe its state representation and round structure.

```

1: Procedure Sponge-FP, pd10*( $M, \nu$ )
2:  $(\overline{M}[1], \dots, \overline{M}[\ell]) \xleftarrow{r} \text{pd10}^*(M)$ ,  $X \leftarrow 0^r \| IV$ 
3: // Absorbing phase
4: for  $i = 1, 2, \dots, \ell$  do
5:   if  $i \neq \ell$  then
6:      $X \leftarrow X \oplus (\overline{M}[i] \| 0^c)$ 
7:   else //  $i = \ell$ 
8:      $X \leftarrow X \oplus (\overline{M}[i] \| C)$ 
9:    $X \leftarrow P(X) \oplus (0^r \| \text{right}_c(X))$ 
10: // Squeezing phase
11:  $\lambda \leftarrow \lceil \frac{\nu}{r'} \rceil$ 
12: for  $i = 1, 2, \dots, \lambda$  do
13:    $Z[i] \leftarrow \text{right}_{r'}(X)$ 
14:   if  $i < \lambda$  then
15:      $X \leftarrow P(X) \oplus (0^r \| \text{right}_c(X))$ 
16: return  $\text{left}_\nu(Z[1] \| \dots \| Z[\lambda])$ 

```

Figure 1: Sponge-F^{P, pd10*}: the construction built upon the sponge-forward construction, with parameters r, c, r' and injective padding pd10^* .

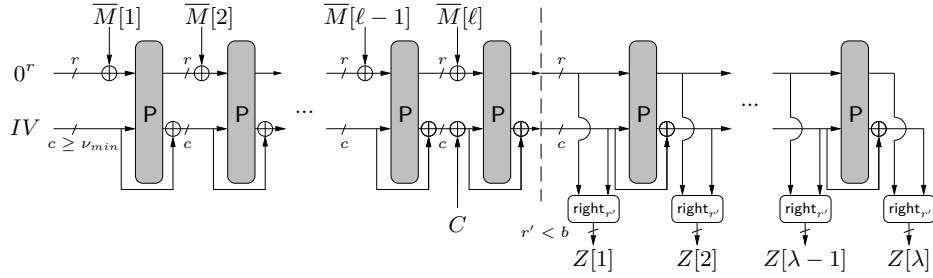


Figure 2: Sponge-with-feed-forward XOF construction $\text{Sponge-F}^{\text{P}, \text{pd10}^*}$, when outputting ν bits ($t = \nu \bmod c$).

2.4.1 State representation

The state of P is represented as a 5×5 array of 64-bit lanes,

$$S = (S[x, y])_{0 \leq x, y < 5} \in (\mathbb{Z}/2^{64}\mathbb{Z})^{5 \times 5}.$$

At the bit level, each lane follows the convention fixed in the notation subsection:

$$S[x, y] = \sum_{z=0}^{63} S[x, y, z] \cdot 2^z, \quad S[x, y, z] \in \{0, 1\}.$$

Whenever arithmetic is performed on the lane indices x and y , the resulting indices are reduced modulo 5.

For convenience, the following notation is used throughout the description of the per-

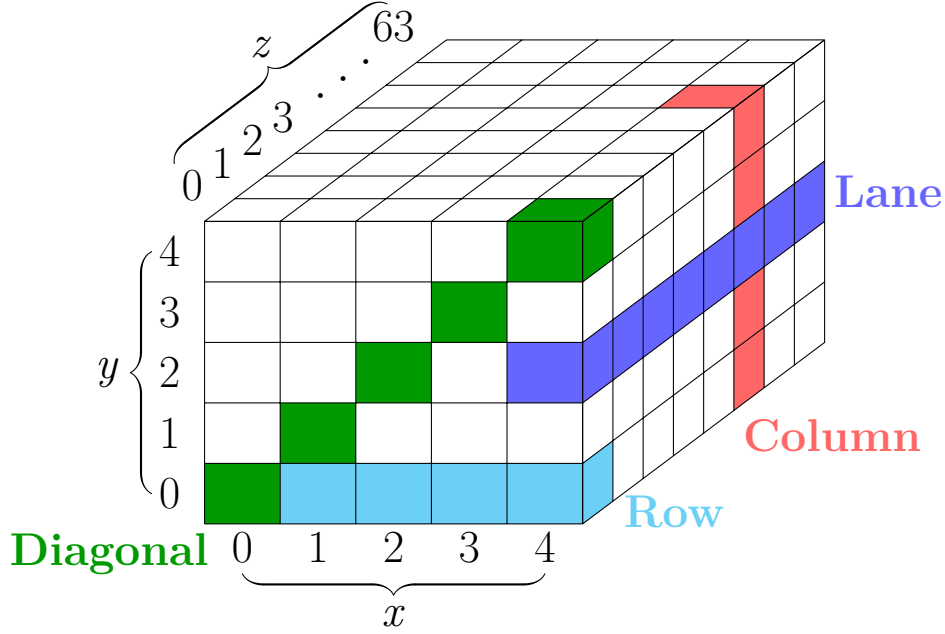


Figure 3: State representation of the Dragon- p .

mutation:

$$\begin{aligned} \text{Lane}_{x,y} &= (S[x, y, 0], \dots, S[x, y, 63]), \\ \text{Row}_y &= (S[0, y], \dots, S[4, y]), \\ \text{Col}_x &= (S[x, 0], \dots, S[x, 4]), \\ \text{Diag}_j &= (S[0, j], S[1, j+1], S[2, j+2], S[3, j+3], S[4, j+4]), \end{aligned}$$

where the second indices in Diag_j are interpreted modulo 5. The organization of the state is illustrated in Figure 3.

2.4.2 Round function

The permutation P consists of 16 rounds:

$$P = R_{15} \circ R_{14} \circ \dots \circ R_0,$$

where

$$R_i = \phi \circ \iota_i.$$

For an input state $S^{(0)}$, the output state is $S^{(16)} = P(S^{(0)})$. Here ι_i denotes the round-constant injection of round i , and ϕ denotes the ARX mixing layer, identical in all rounds.

2.4.3 Round-constant injection

For each round $i \in \{0, 1, \dots, 15\}$, the mapping ι_i XORs a 64-bit round constant RC_i into lane $S[0, 0]$ and leaves all other lanes unchanged. Formally, ι_i is defined lane-wise by

$$\iota_i(S)[x, y] = \begin{cases} S[0, 0] \oplus \text{RC}_i, & \text{if } (x, y) = (0, 0), \\ S[x, y], & \text{otherwise.} \end{cases}$$

The constants are listed in Table 3. They are written as 64-bit hexadecimal words.

Table 3: Round constants of the Dragon- p .

Round i	RC_i	Round i	RC_i
0	0x000000000000000F	8	0x0000000000000087
1	0x000000000000001E	9	0x0000000000000096
2	0x000000000000002D	10	0x00000000000000A5
3	0x000000000000003C	11	0x00000000000000B4
4	0x000000000000004B	12	0x00000000000000C3
5	0x000000000000005A	13	0x00000000000000D2
6	0x0000000000000069	14	0x00000000000000E1
7	0x0000000000000078	15	0x00000000000000F0

2.4.4 The ARX mixing layer

The mixing layer ϕ consists of two transformations, **Diag** and **Col**, defined as

$$\phi = \text{Col} \circ \text{Diag}.$$

Both transformations are built from the five-word ARX permutation **UPDATE**. The transformation **Diag** applies **UPDATE** to five cyclic diagonals of the state, while **Col** applies **UPDATE** to the five columns of the resulting state.

The update transformation. The transformation **UPDATE** is the basic building block of the mixing layer and operates on five 64-bit words:

$$\text{UPDATE} : (\mathbb{Z}/2^{64}\mathbb{Z})^5 \rightarrow (\mathbb{Z}/2^{64}\mathbb{Z})^5.$$

Given an input tuple (a, b, c, d, e) , the five words are updated sequentially as follows:

$$\begin{aligned} b &\leftarrow b \boxplus e, & d &\leftarrow (d \lll r_d) \oplus b, \\ a &\leftarrow a \boxplus d, & c &\leftarrow (c \lll r_c) \oplus a, \\ e &\leftarrow e \boxplus c, & b &\leftarrow (b \lll r_b) \oplus e, \\ d &\leftarrow d \boxplus b, & a &\leftarrow (a \lll r_a) \oplus d, \\ c &\leftarrow c \boxplus a, & e &\leftarrow (e \lll r_e) \oplus c. \end{aligned}$$

The resulting values are denoted by

$$(a', b', c', d', e') = \text{UPDATE}(a, b, c, d, e).$$

The structure of **UPDATE** transformation is illustrated in Figure 4 and the rotation offsets are fixed to $(r_a, r_b, r_c, r_d, r_e) = (1, 10, 24, 33, 49)$.

Diagonal and column updates. Let S be the input state of the mixing layer. The intermediate state T and the output state U are defined by

$$\text{Diag}_j(T) = \text{UPDATE}(\text{Diag}_j(S)), \quad 0 \leq j < 5,$$

and

$$\text{Col}_x(U) = \text{UPDATE}(\text{Col}_x(T)), \quad 0 \leq x < 5,$$

respectively, where all indices are reduced modulo 5. The output of the mixing layer is $\phi(S) = U$.

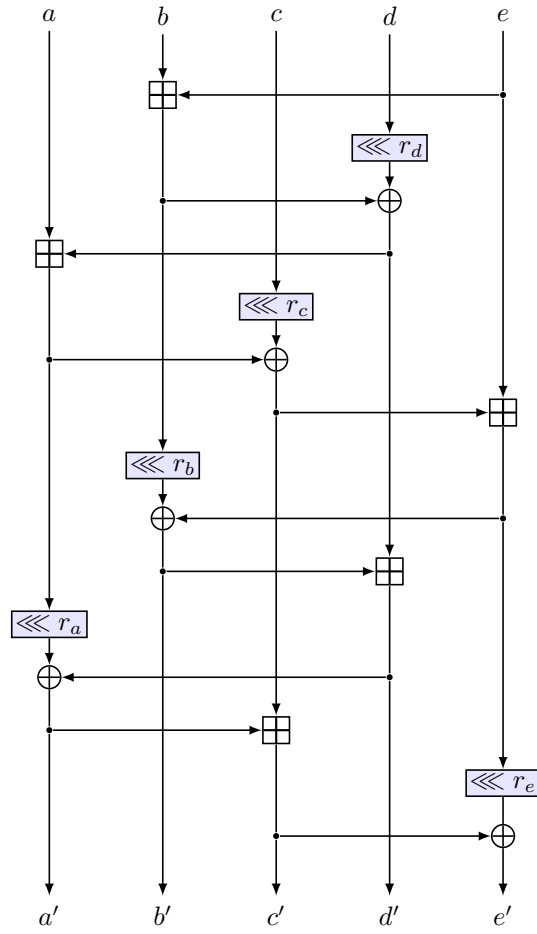


Figure 4: The UPDATE transformation of the Dragon- p .

2.4.5 Invertibility

The permutation **Dragon- p** is invertible. The round-constant injection ι_i is invertible because XOR with a fixed constant is self-inverse. Likewise, the transformation **UPDATE** is invertible since it is composed solely of modular additions, bitwise XOR operations, and fixed cyclic rotations, all of which are invertible.

Consequently, the diagonal update stage **Diag**, the column update stage **Col**, the mixing layer $\phi = \text{Col} \circ \text{Diag}$, and every round transformation $R_i = \phi \circ \iota_i$ are invertible. Therefore, the Dragon permutation

$$P = R_{15} \circ R_{14} \circ \cdots \circ R_0$$

is a bijection on the 1600-bit state space.

2.5 The hash: Dragon

This subsection instantiates the sponge-forward mode with the Dragon permutation and specifies the Dragon hash and XOF computations at the lane level. It defines the instance parameters, the placement of initialization vectors (IVs) and message words in the state, the final-block processing rule, and the output extraction order.

2.5.1 Instance parameters

For each Dragon instance, the state size is $b = 1600$ bits and the underlying primitive is the 16-round Dragon permutation specified above. The instance-dependent parameters are the rate r , the capacity c , the output length, and a public initialization vector. Since the rate and capacity are lane-aligned, we write

$$n_c = c/64, \quad n_r = r/64$$

for the corresponding numbers of 64-bit capacity and rate lanes. The parameters of the fixed-output instances and the XOF interface are given in Table 4.

Table 4: Instance parameters for Dragon.

Instance	c	r	n_c	n_r	Output length
Dragon-512	576	1024	9	16	512
Dragon-768	832	768	13	12	768
Dragon-1024	1088	512	17	8	1024
Dragon-XOF-256	576	1024	9	16	ν
Dragon-XOF-384	832	768	13	12	ν
Dragon-XOF-512	1088	512	17	8	ν

Initialization vectors. Each instance is initialized with a public IV consisting of n_c 64-bit lanes, denoted as $(IV_0, \dots, IV_{n_c-1})$. They are generated in a transparent counter mode from SM3. For a fixed-output instance, the input string to the generator is

$$\text{Dragon-}h \parallel \text{IV} \parallel \mathbf{r} = r \parallel \mathbf{c} = c \parallel,$$

where h is the digest length and r, c are the rate and capacity. The string is concatenated with a 32-bit big-endian counter, hashed with SM3, and the resulting 256-bit blocks are concatenated until c IV bits are obtained. For an XOF instance, the domain string additionally includes the squeezing rate r' :

$$\text{Dragon-XOF-}t \parallel \text{IV} \parallel \mathbf{r} = r \parallel \mathbf{c} = c \parallel \mathbf{r}' = r' \parallel,$$

where t specifies the target security level of the XOF construction. The IV is generated using the same SM3-based counter construction and truncated to c bits.

This construction ensures domain separation across all parameter sets, avoids all-zero initialization of the capacity portion, and yields reproducible IVs. The resulting IV values are listed in Table 5, and their injection into the state is defined in the next subsection.

Table 5: Public IV lanes for the Dragon hash and XOF instances.

Instance	Index	IV value	Index	IV value
Dragon-512	IV_0	0x0B6B2E774D8F675B	IV_5	0x86F2A0CA3B8E0D1E
	IV_1	0x4402CE26DD9509A5	IV_6	0xD8F5D940D993F0D3
	IV_2	0x9B39251883C81294	IV_7	0xDDAA68D459FF4EE7
	IV_3	0x8760145EFF9D8EB9	IV_8	0x2D5EBD181E489EBB
	IV_4	0xDC1851BCA4509C55	—	—
Dragon-768	IV_0	0xB92F84BD6C40F304	IV_7	0xBE2C8765CD92A3C3
	IV_1	0x7BB16442F4EBC689	IV_8	0x70BC9161408A1782
	IV_2	0xEB378CB85BED63B5	IV_9	0x7FA4596963C70DE3
	IV_3	0x29027D5269E5CA89	IV_{10}	0x586AD4AC86CB8A14
	IV_4	0x722ED7D4C36F2A8B	IV_{11}	0x5FA7A2FF01C584DE
	IV_5	0xC2BD1577105B5097	IV_{12}	0x3F27FDC10E096F1E
	IV_6	0xA84360373179EE75	—	—
Dragon-1024	IV_0	0x669A8E1412CC3320	IV_9	0x2DCD8C5F294403FA
	IV_1	0xC867C18B5C7C055F	IV_{10}	0xA98B10BDEE3BA4FA
	IV_2	0xB5A2FFC3B0EAE1C4	IV_{11}	0x96F93E8D3CAAE500
	IV_3	0x24D1C3E7CF0DD2BA	IV_{12}	0x72153ED182F6EBFB
	IV_4	0xBC3EB70D5574E8D7	IV_{13}	0x635EB7B10580D109
	IV_5	0x8FF1F9CEE145AED7	IV_{14}	0x1701B3BAB89C5A08
	IV_6	0x6332AC77291C10BF	IV_{15}	0x9131EE6B18889170
	IV_7	0x14311D2F70498F2D	IV_{16}	0xC63471C8A9B1702A
	IV_8	0xFDD3B4364B999591	—	—
Dragon-XOF-256	IV_0	0x1C0B2988E548FD1D	IV_5	0x526274D2621E2B08
	IV_1	0x053698B7CC4A88C9	IV_6	0xFE7246AC355FB602
	IV_2	0x2193B7DDE2D2BB6C	IV_7	0xDECFAAE131C61108
	IV_3	0x78543EC346B92599	IV_8	0xFAC725872F599037
	IV_4	0xA005BC1C28A981A5	—	—
Dragon-XOF-384	IV_0	0x1088EE6AD2B51737	IV_7	0xA7258DF0B17738F7
	IV_1	0x293AF3A5DA8528C1	IV_8	0x5BDBB762065C05B5
	IV_2	0xC00B49341758E6BB	IV_9	0x72EBF9F144CB0096
	IV_3	0x74F20A039F6D0274	IV_{10}	0x7BC790700801D690
	IV_4	0x8041B7E7E2135169	IV_{11}	0xE927F45943C2BC11
	IV_5	0xEA355D23851D0F07	IV_{12}	0x01BBFE4B907A4A80
	IV_6	0x66EAD0416DE0FCA	—	—
Dragon-XOF-512	IV_0	0xBD52175FC753D3D4	IV_9	0x6F248BBD94D4B9B9
	IV_1	0xBF62000ACB12405F	IV_{10}	0x55F7FADCD1110E22
	IV_2	0x519343F95981BBCE	IV_{11}	0xCDBEBA91248FD4E3
	IV_3	0xF0E203D2561601C9	IV_{12}	0xF68F22D737F1EA69
	IV_4	0xF33330E05F38A3C2	IV_{13}	0xC1A02AEEB836F700
	IV_5	0x73D6F28EEC922318	IV_{14}	0x32CADA2F882C6A3
	IV_6	0xF02BFB02360C2345	IV_{15}	0x077B759252B1C628
	IV_7	0xBC2CD1D926C6C690	IV_{16}	0xD3B4A68FC9CC970F
	IV_8	0xEA2D075670D4FD64	—	—

2.5.2 Message layout and state loading

To describe initialization and absorption, we use the linear lane index

$$\ell(x, y) = x + 5y, \quad 0 \leq x, y < 5.$$

Thus, the state lanes are ordered from $S[0, 0]$ to $S[4, 4]$ as $\ell(x, y)$ increases from 0 to 24.

At any point during the computation, the capacity portion of the state consists of the n_c lanes with the smallest linear indices, while the remaining n_r lanes form the rate portion. During initialization, the IV lanes are loaded into the capacity portion in ascending order of $\ell(x, y)$, whereas all rate lanes are set to zero:

$$S[x, y] = \begin{cases} IV_{\ell(x, y)}, & 0 \leq \ell(x, y) < n_c, \\ 0, & n_c \leq \ell(x, y) < 25. \end{cases}$$

This initialization is performed only once. For all subsequent message blocks, the capacity portion retains the state produced by the previous processing step and is not reloaded with the IV. Message words are mapped exclusively to the rate portion according to the layout defined below.

Each rate block is represented as a sequence of n_r 64-bit words,

$$M' = M_{n_r-1} \| M_{n_r-2} \| \cdots \| M_0,$$

where M_{n_r-1} is the most significant word of the block. The message words are associated with the n_r highest-indexed lanes of the state in increasing lane-index order. More precisely, for $0 \leq i < n_r$, let

$$k_i = 25 - n_r + i, \quad x_i = k_i \bmod 5, \quad y_i = \lfloor k_i/5 \rfloor.$$

Then M_i is associated with lane $S[x_i, y_i]$. Equivalently, M_0 is associated with the lowest-indexed rate lane, whereas M_{n_r-1} is associated with the highest-indexed state lane.

Example (Dragon-512). For Dragon-512, the capacity and rate parameters are $c = 576$ and $r = 1024$, corresponding to $n_c = 9$ capacity lanes and $n_r = 16$ rate lanes. Consequently, the capacity portion occupies the nine lowest-indexed state lanes,

$$S[0, 0], S[1, 0], \dots, S[4, 0], S[0, 1], S[1, 1], S[2, 1], S[3, 1],$$

which are initialized with $IV_0, IV_1, \dots, IV_8$, respectively.

After padding, a message block is represented as

$$M' = M_{15} \| M_{14} \| \cdots \| M_0.$$

The sixteen message words occupy the sixteen highest-indexed state lanes, with M_{15} mapped to $S[4, 4]$, M_{14} mapped to $S[3, 4]$, and M_0 mapped to $S[4, 1]$.

Figure 5 illustrates the placement of the IV and message words within the state.

2.5.3 Padding and final block processing

Let M be the input message and let $r = 64n_r$ be the rate. The message is processed in successive r -bit blocks. Each complete block is loaded into the rate portion of the state according to the mapping rule defined in the previous subsection. After the absorption of each complete block, a sponge-forward step is performed.

After all complete blocks have been processed, the remaining suffix M_{rem} is padded using the same $\text{pd}10^*$ rule as in the sponge-forward mode:

$$\text{pd}10^*(M) = M \| 10^{r - ((|M|+1) \bmod r)}.$$

Equivalently, the padding is applied to M_{rem} as follows. Let $t = |M_{\text{rem}}|$, where $0 \leq t < r$.

- If $0 < t < r - 1$, the final r -bit block is

$$M_{\text{rem}} \| 1 \| 0^{r-t-1}.$$

$y = 4$	M_{11}	M_{12}	M_{13}	M_{14}	M_{15}
$y = 3$	M_6	M_7	M_8	M_9	M_{10}
$y = 2$	M_1	M_2	M_3	M_4	M_5
$y = 1$	IV_5	IV_6	IV_7	IV_8	M_0
$y = 0$	IV_0	IV_1	IV_2	IV_3	IV_4
	$x = 0$	$x = 1$	$x = 2$	$x = 3$	$x = 4$

Figure 5: State initialization and message-word placement for Dragon-512. The capacity portion is initialized with $IV_0, \dots, IV_8$, while the 16 message words occupy the rate portion from M_0 to M_{15} .

- If $t = r - 1$, the bit 1 exactly fills the current rate block. This block is processed as

$$M_{\text{rem}} \| 1,$$

and an additional final block 0^r is then processed.

- If $t = 0$, that is, if $|M|$ is already a multiple of the rate, an additional final block

$$1 \| 0^{r-1}$$

is processed.

With the message layout defined in the previous subsection, the bit 1 introduced by the 10^* rule is placed immediately after the final message bit, and all remaining less-significant positions of the corresponding rate block are set to zero.

The padded block(s) are loaded into the rate portion of the state in the same manner as ordinary message blocks. The final-domain constant $C = 0^{c-1} \| 1$ is applied only to the last processed block. In the lane representation this corresponds to XORing the constant 1 into the least significant bit of lane $S[0, 0]$:

$$S[0, 0] \leftarrow S[0, 0] \oplus 1.$$

Thus, in the second case above, the block $M_{\text{rem}} \| 1$ is not domain-separated; the domain-separation bit is applied to the following all-zero block 0^r . A sponge-forward step is performed after each processed block, and the final state is the state obtained after the last such step.

2.5.4 Digest extraction

For the Dragon-XOF instances, the variable-length output is produced by the squeezing phase of Fig. 1, which emits r' bits per permutation call with the instance parameters of Table 6, and the result is truncated to its first ν bits. The remainder of this subsection specifies the extraction order for the fixed-output hash instances.

For a fixed-output instance with digest length d , let $n_d = d/64$ denote the number of output lanes. The digest is extracted from the n_d lowest-indexed lanes of the final state. For $0 \leq j < n_d$, define

$$x_j = j \bmod 5, \quad y_j = \lfloor j/5 \rfloor.$$

The digest lane with index j is

$$D_j = S[x_j, y_j].$$

The digest is obtained by serializing these lanes in decreasing lane-index order:

$$D = D_{n_d-1} \| D_{n_d-2} \| \cdots \| D_0,$$

where each lane is encoded as an eight-byte string in most-significant-byte-first order.

Example (Dragon-512). For Dragon-512, $d = 512$ and therefore $n_d = 8$. The extracted lanes are

$$S[0, 0], S[1, 0], S[2, 0], S[3, 0], S[4, 0], S[0, 1], S[1, 1], S[2, 1].$$

Consequently, the digest is

$$D = S[2, 1] \| S[1, 1] \| S[0, 1] \| S[4, 0] \| S[3, 0] \| S[2, 0] \| S[1, 0] \| S[0, 0].$$

2.6 Extendable-Output Function (XOF)

A feature of **Dragon** is that the same sponge-forward construction supports a variable output length. Instantiating **Sponge-F^{P, pd10*}** with a chosen capacity gives an extendable-output function. On input (M, ν) it returns the first ν bits of an arbitrarily long output, squeezed r' bits per permutation call as the rightmost r' bits of the state. The construction is given in Fig. 1. We provide three instances, **Dragon-XOF-256**, **Dragon-XOF-384**, and **Dragon-XOF-512**. Their parameters are listed in Table 6.

Table 6: The three **Dragon** extendable-output functions. The security level is the target $\log_2 Q$; r and c are the absorbing rate and capacity with $r + c = 1600$; r' is the squeezing rate, the number of output bits produced per permutation call during squeezing.

Instance	Security ($\log_2 Q$)	r	c	r'
Dragon-XOF-256	256	1024	576	1280
Dragon-XOF-384	384	768	832	1152
Dragon-XOF-512	512	512	1088	1024

Because the feed-forward lets the squeezing rate r' exceed the absorbing rate r , each squeezing call already produces r' bits of output. The c -bit *IV* values of the three XOF instances are listed, together with the hash *IVs*, in Table 5.

2.7 Security claim

We state the security claims for the **Dragon** family as design targets. The claims hold for messages of length up to 2^{64} bits. These targets follow the security of the **Sponge-F^{P, pd10*}** mode established by Guo et al. [GHJ+25] in the random-permutation model.

Fixed-output hashing. Each **Dragon- h** instance targets $h/2$ -bit collision security and h -bit preimage security. The feed-forward yields $\min\{c, h\} = h$ -bit preimage security, since $c > h$ for every instance. Concretely, **Dragon-512** targets 256-bit collision and 512-bit preimage security, **Dragon-768** targets 384-bit collision and 768-bit preimage security, and **Dragon-1024** targets 512-bit collision and 1024-bit preimage security. The corresponding instance parameters are listed in Table 4.

Extendable output. The extendable-output instances Dragon-XOF-256, Dragon-XOF-384, and Dragon-XOF-512 target a generic security level of $\log_2 Q = 256, 384,$ and 512 bits, respectively, where Q is the number of permutation queries. These levels are given in Table 6.

3 Design Rationale

3.1 The mode

The sponge construction of Bertoni et al. [BDPVA07, BDPVA11] is a popular approach for cryptographic hashing. At a high level, the sponge operates on a state of size b bits, which is split into an inner part of size c bits (the capacity) and an outer part of size r bits (the rate), where $b = c + r$. The sponge consists of an absorbing phase and a squeezing phase. In the absorbing phase, data is compressed into the state r bits at a time, interleaved with an evaluation of a b -bit permutation P . In the squeezing phase, a digest is extracted from the state r' bits at a time (possibly $r' \neq r$, as proposed by [GPP11]), again interleaved with an evaluation of P . The sponge found quick adoption right after its introduction. In particular, Bertoni et al. designed the sponge instance KECCAK, which has become the final winner of the NIST SHA-3 competition [D⁺15]. It was quickly acknowledged that the sponge was particularly well-suited for lightweight hashing [BLP⁺08, AHMN10, BKL⁺11, GPP11], and the winner of NIST Lightweight Cryptography competition Ascon-Hash [STMC⁺24] is also a sponge-based algorithm.

To support its security, the designers proved that the sponge construction is indistinguishable from a random oracle up to $2^{c/2}$ adversarial queries [BDPV08]. This implies collision security up to $\min\{2^{c/2}, 2^{h/2}\}$ queries and (second) preimage security up to $\min\{2^{c/2}, 2^h\}$ queries [AMP11], and the preimage security was later improved to $\min\{\max\{2^{h-r'}, 2^{c/2}\}, 2^h\}$ queries by a dedicated proof [LM22]. The latter bound approaches h when r' is very small or $c \approx 2h$. As shown by attacks [BDPVA07, LM22], both the $\min\{\max\{2^{h-r'}, 2^{c/2}\}, 2^h\}$ bit preimage security and the $c/2$ bit second preimage security are roughly tight. Due to this, a sponge-based hashing aiming at h -bit output, $h/2$ bit collision security and $\approx h$ bit (second) preimage security has to set $c \geq 2h$: indeed, this is the choice of the SHA-3 family (SHA3- h with $h \in \{224, 256, 384, 512\}$ defining the output size sets the capacity $c = 2h$). Facing the 1024-bit (second) preimage security requirement of Chinese calls for new generation cryptographic algorithms [NIC25], this constitutes a severe limitation: it means one has to set $c = 2048$ and design a secure permutation with width greater than 2048-bit, which is quite difficult.

It is easy to see that the meet-in-the-middle-style preimage attack in [LM22] crucially relies on a sort of *invertibility* in sponge: given a b -bit intermediate state, one can both invoke P to evaluate forwardly and invoke P^{-1} to evaluate backwardly in the construction. If a *non-invertible* random function is used instead of a random permutation, then Foekens [Foe23] argues that the resulting sponge variant enjoys optimal $\min\{c, h\}$ bit (second) preimage security. By this, a “cryptographically strong” non-invertible function with a width of e.g., 1600 bits, suffices for the required 1024-bit (second) preimage security. However, it is rather non-trivial to design a “cryptographically strong” non-invertible function.

On the other hand, if we *add feed-forward back* to the permutation-based sponge, then we obtain a sponge variant built upon the *permutation-based Davies-Meyer construction* $DM^P(X) := P(X) \oplus X$. While the Davies-Meyer construction does not behave like a random function [CDMP05], it is both *non-invertible* and of some cryptographic strength, and might help achieve the 1024-bit (second) preimage security from a e.g., 1600-bit permutation.

We develop this idea into the *sponge-forward XOF construction* $\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ [GHJ⁺25], which Dragon adopts as its mode of operation, as illustrated in Fig. 2. For each permutation-call we consider *feeding the c -bit inner part* of the input forward to the output (feeding the outer part forward does not increase security). Moreover, in the squeezing phase, the output is squeezed at the rate of r' bits each time, taken as the rightmost r' bits of the state, as shown in Fig. 2 (Top). As shown by Andreeva et al. [AMP12], outputting the inner part may cause length-extension attacks. To prevent, when the last padded message block $\overline{M}[\ell]$ is XORed into the outer part of the state, a constant $C = 0^{c-1}||1$ is simultaneously XORed into the inner part. This borrows ideas from the Merkle-Damgård with permutation construction [HPY12].

Concurrently and independently, Sun et al. [SLZ⁺25] developed the same Davies-Meyer feed-forward idea into their SPONGE-DM and SPONGE-EDM constructions, which achieve comparable h -bit preimage and $\min\{h, c - \log_2 \ell\}$ -bit second-preimage security from a b -bit permutation.

In fact, this idea achieves three gains simultaneously:

- Its collision and (second) preimage security can be reduced to well-defined, simple properties of the underlying permutation;
- It achieves beyond $b/2$ bit (second) preimage security from a b -bit permutation;
- It achieves meaningful quantum collision and (second) preimage security bounds.

As results, let $\ell_{\max}$ be the maximum number of blocks in allowed message inputs. As proven in [GHJ⁺25], when $r = b - c$ is not too small and output length $h \leq c - \log_2 \ell_{\max}$, $\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ ensures:

1. $h/2$ bits collision security, h bits preimage security, and $\min\{h, c - \log_2 \ell_{\max}\}$ bits second preimage security;
2. $h/4$ bits quantum collision security, $h/2$ bits quantum preimage security, and $\min\{\frac{h}{2}, \frac{c - \log_2 \ell_{\max}}{2}\}$ bits quantum second preimage security;
3. Approximately $\min\{2^{c/2}, 2^{b-c}\}$ bits indistinguishability security.

The preimage, second preimage and quantum security bounds are much better than those of the standard sponge. By these, to support messages up to 2^{64} bits, we can conclude on the parameter choices as follows:

1. To meet the 512-bit (second) preimage security requirement of Chinese calls for new generation cryptographic algorithms, one need to set $h = 512$ and $c = 512 + 64 = 576$. When $b \geq 832$, this achieves 256-bit indistinguishability security.
2. To meet the 1024-bit (second) preimage security requirement of Chinese calls for new generation cryptographic algorithms, one need to set $h = 1024$ and $c = 1024 + 64 = 1088$. When $b \geq 1600$, this achieves 512-bit indistinguishability security.
3. To meet the 768-bit (second) preimage security requirement of Chinese calls for new generation cryptographic algorithms, one need to set $h = 768$ and $c = 768 + 64 = 832$. When $b \geq 1216$, this achieves 384-bit indistinguishability security.

Since the output length h is less than c in these cases, $\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ simply outputs the h bit inner part of the final state, as shown in Fig. 6.

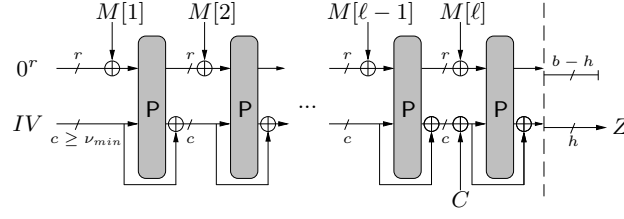


Figure 6: Simplified illustration for the conventional fixed-output-length hash with output length $h \leq c$.

3.2 The permutation

The analysis of the sponge-forward mode reduces the collision and (second) preimage security of $\text{Sponge-F}^{\text{P}, \text{pd}10^*}$ to simple, well-defined properties of the underlying b -bit permutation P , modeled as a random permutation. The permutation $\text{Dragon-}p$ is designed to justify this idealization. It targets the behavior of a random 1600-bit permutation that exhibits no exploitable structural distinguisher. The width $b = 1600$ is the smallest one that simultaneously meets the 1024-bit (second) preimage requirement, which fixes $c = 1088$, and the 512-bit indistinguishability requirement, which fixes $r = b - c = 512$. A single 1600-bit permutation therefore instantiates every member of the family.

State geometry and ARX operations. The $\text{Dragon-}p$ follows a word-oriented design. Its 1600-bit state is organized as a 5×5 array of 64-bit lanes $S[x, y]$, where $0 \leq x, y < 5$, matching the state geometry of KECCAK [D⁺15]. This preserves a well-studied state organization while allowing each lane to be represented as a native 64-bit machine word for efficient software implementation.

The main departure from KECCAK lies in the nonlinear layer. Instead of a bit-oriented S-box layer, $\text{Dragon-}p$ employs the 64-bit ARX UPDATE transformation, composed solely of additions modulo 2^{64} , cyclic rotations, and XOR operations. Modular addition introduces nonlinearity through carry propagation, while rotations redistribute bits within each word and XOR operations efficiently combine information between words. Consequently, the round function is table-free, constant-time, and well suited to high-throughput software implementations. These are universal integer operations present on every general-purpose processor. $\text{Dragon-}p$ thus attains its throughput with no dependence on any dedicated cryptographic instruction-set extension, such as Intel’s AES-NI or SHA Extensions, or the Armv8 Cryptographic Extension. This is a deliberate departure from designs whose competitiveness is tied to the availability of such vendor-specific instructions on the target machine: $\text{Dragon-}p$ is portable by construction and delivers stable performance across platforms and instruction sets.

Diagonal and column diffusion. The mixing layer combines diagonal and column interactions. It first applies the UPDATE transformation independently to the five cyclic diagonals, creating dependencies across both row and column dimensions. The same transformation is then applied independently to the five columns, further mixing the words brought together by the diagonal stage. The two mixing stages complement each other. This two-stage organization enables local word-level differences to propagate rapidly throughout the entire state, achieving full diffusion within two rounds while maintaining a simple and regular round structure.

Choice of rotation offsets. The rotation offsets do not provide nonlinearity by themselves, but they determine how the carry effects of modular addition are redistributed

among bit positions and words. We therefore selected them by a diffusion-oriented search whose goal was to avoid simple structural relations and to obtain fast avalanche behavior in the ARX update.

The search first restricted the five offsets to separated ranges

$$r_a \in [1, 9], \quad r_b \in [10, 19], \quad r_c \in [20, 29], \quad r_d \in [30, 39], \quad r_e \in [40, 49].$$

This reduces the exhaustive-search space and prevents all rotations from acting at similar bit distances. We then applied an algebraic pruning step to remove candidates with obvious regularities, such as large common divisors with the word size, unbalanced parity patterns, complementary offsets, simple multiple relations, or a zero total shift modulo 64.

The remaining candidates were evaluated in two stages. First, a linearized version of the UPDATE transformation was used to assess the diffusion properties of the rotation-based structure, in which all modular additions were replaced by XOR operations. This removes carry propagation effects and allows the analysis to focus on the underlying word-level mixing induced by rotations and XORs. Candidates that performed well under this conservative model were then evaluated under the full ARX UPDATE transformation. This evaluation sampled single-bit differences over random backgrounds across several rounds. The scoring criteria included the deviation from ideal avalanche behavior, diffusion uniformity across the state, and the presence of low-activity input positions.

Consequently, the selected rotation offsets are

$$(r_a, r_b, r_c, r_d, r_e) = (1, 10, 24, 33, 49).$$

This tuple remained consistently among the best candidates across repeated runs, providing fast and stable diffusion in both the linearized model and the full ARX-based evaluation. These measurements are used solely as design heuristics for selecting rotation offsets and formal differential and linear trail analyses are presented in Section 4.2.3.

Choice of round constants. The round constants are injected into a single 64-bit lane before the mixing layer. They are not intended to provide diffusion or nonlinearity, which are fully handled by the ARX-based permutation. Instead, they serve to break round symmetry and prevent trivial structural properties such as slide, rotational, or invariant-subspace symmetries.

The constants follow the simple arithmetic rule

$$\text{RC}_i = 15(i + 1), \quad 0 \leq i < 16,$$

ensuring nonzero and pairwise distinct values. This simple structure allows on-the-fly generation by iterative addition of `0x0f`, eliminating the need for a lookup table and requiring only a single XOR per round, which makes the design implementation-friendly.

Choice of the number of rounds. The number of rounds is chosen to balance security margin and software performance. Each additional round of the ARX-based permutation incurs a direct performance cost, while too few rounds may leave the design within the range where reduced-round structural distinguishers remain effective.

To determine an appropriate security margin, we rely on the reduced-round analyses in Section 4.2. The differential and linear trail searches show a sharp increase in trail weight from one to two rounds. In addition, the differential-linear analysis further shows that single-bit correlations fall below the empirical detection threshold within three rounds, indicating that exploitable statistical structure disappears quickly.

Importantly, these observations are not intended to provide full-round security bounds, but rather to characterize the decay of reduced-round distinguishers. Based on this

trend, we set the full permutation to 16 rounds, which places the design well beyond the range where such distinguishers remain effective, while maintaining efficiency for high-throughput software implementations.

3.3 Design summary

Overall, the design of Dragon separates the roles of the mode and the permutation.

- The mode is responsible for providing the overall security level and accommodating the target preimage and second-preimage requirements, while also enabling instance separation through SM3-derived initialization values.
- The permutation Dragon- p operates on a compact 5×5 lane structure and is designed for efficient constant-time software implementation using 64-bit ARX operations. It achieves rapid diffusion through diagonal-and-column mixing, while rotation offsets and round constants are selected to support diffusion and symmetry breaking with low implementation cost.

With 16 rounds, the full permutation lies well beyond the range where reduced-round differential, linear, and differential-linear distinguishers remain effective, while preserving the efficiency of the word-oriented design.

4 Security Analysis

4.1 Security analysis of the mode

4.1.1 Security notions

Collision and (Second) Preimage Security This paragraph presents the traditional security definitions for permutation oracle-based fixed-output-length hash functions h^P with h -bit output. First, given an adversary $\mathcal{A}^{P, P^{-1}}$, we define the attack advantage of $\mathcal{A}^{P, P^{-1}}$ against the collision security of h^P as follows:

$$\mathbf{Adv}_h^{\text{coll}}(\mathcal{A}) := \Pr_{P, \varepsilon} [\mathcal{A}^{P, P^{-1}} \Rightarrow (M, M') : M \neq M', h^P(M) = h^P(M')], \quad (2)$$

where the probability is taken over the choice of P (if P has randomness) and the randomness ε of $\mathcal{A}^{P, P^{-1}}$.

Finding a preimage for h^P requires finding $M \in \{0, 1\}^*$ for a challenge image $Z \in \{0, 1\}^h$, such that $h^P(M) = Z$. Given h^P and $\mathcal{A}^{P, P^{-1}}$ and a challenge image $Z \in \{0, 1\}^h$, we define the advantage of $\mathcal{A}^{P, P^{-1}}$ against the Z -preimage security of h^P as follows:

$$\mathbf{Adv}_h^{Z\text{-pre}}(\mathcal{A}) := \Pr_{P, \varepsilon} [\mathcal{A}^{P, P^{-1}}(Z) \Rightarrow M \in \{0, 1\}^*, h^P(M) = Z]. \quad (3)$$

Then, the advantage of $\mathcal{A}^{P, P^{-1}}$ against the *everywhere preimage* security of h^P is defined by taking maximum over all image Z .

$$\mathbf{Adv}_h^{\text{epre}}(\mathcal{A}) := \max_{Z \in \{0, 1\}^h} \left\{ \mathbf{Adv}_h^{Z\text{-pre}}(\mathcal{A}) \right\}. \quad (4)$$

Finding a second preimage for h^P for a challenge message $M \in \{0, 1\}^*$ requires finding $M' \in \{0, 1\}^*$ such that $M' \neq M$ while $h^P(M') = h^P(M)$. Concretely, given $\mathcal{A}^{P, P^{-1}}$ and a challenge message $M \in \{0, 1\}^*$, we define the advantage of $\mathcal{A}^{P, P^{-1}}$ against the M -second preimage security of h^P as follows:

$$\mathbf{Adv}_h^{M\text{-spr}}(\mathcal{A}) := \Pr_{P, \varepsilon} [\mathcal{A}^{P, P^{-1}}(M) \Rightarrow M' \in \{0, 1\}^*, M' \neq M, h^P(M') = h^P(M)]. \quad (5)$$

Typically, second preimage resistance is then defined w.r.t. the bit-length of the challenge message M [RS04, Yas08], which influences concrete security (as observed in [KS05, SLZ⁺25]). Formally, the advantage of $\mathcal{A}^{\mathbf{P}, \mathbf{P}^{-1}}$ against the everywhere second preimage security of $\mathbf{h}^{\mathbf{P}}$ for bl -bit challenge messages is:

$$\mathbf{Adv}_{\mathbf{h}}^{\text{espr}[bl]}(\mathcal{A}) := \max_{M \in \{0,1\}^{bl}} \left\{ \mathbf{Adv}_{\mathbf{h}}^{M\text{-spr}}(\mathcal{A}) \right\}. \quad (6)$$

As discussed in [GHJ⁺25, Sect. 2.2], everywhere-style notions cannot be security goals nor assumptions for non-idealized, keyless hash functions. For keyless hash functions, Rogaway and Shrimpton [RS04] introduced *always preimage resistance*, which requires $\mathcal{A}(Z)$ to find the preimage for $Z = \mathbf{h}(M)$ for a uniformly picked message $M \xleftarrow{\$} \{0,1\}^{bl}$. This is called *domain-oriented preimage resistance* in [AS11]. As its dual, *range-oriented preimage resistance* [AS11] requires $\mathcal{A}^{\mathbf{P}, \mathbf{P}^{-1}}(Z)$ to find the preimage for a uniformly picked image $Z \xleftarrow{\$} \{0,1\}^h$. Andreeva and Stam [AS11] further extended these notions to *distribution-oriented*, by requiring the challenge image Z to be sampled according to a distribution $\mathbf{d}$ (which then becomes a parameter of the security definition). In the keyless setting, by allowing the distribution to depend on the function $\mathbf{h}$, domain-distribution-oriented and range-distribution-oriented variants can be unified: a distribution over $\{0,1\}^*$ can be transformed into an $\mathbf{h}$ -dependent distribution over $\{0,1\}^h$.

Concretely, (in the plain model,) given a distribution $\mathbf{d}$ over $\{0,1\}^h$, the advantage of $\mathcal{A}$ against the $\mathbf{d}$ -oriented preimage security of $\mathbf{h}$ is

$$\begin{aligned} \mathbf{Adv}_{\mathbf{h}}^{\mathbf{d}\text{-pre}}(\mathcal{A}) &:= \Pr_{\varepsilon} [Z \xleftarrow{\$} \mathbf{d}, \mathcal{A}(Z) \Rightarrow M \in \{0,1\}^*, \mathbf{h}^{\mathbf{P}}(M) = Z] \\ &= \sum_{Z \in \{0,1\}^h} \Pr[Z^* \xleftarrow{\$} \mathbf{d} : Z^* = Z] \cdot \mathbf{Adv}_{\mathbf{h}}^{Z\text{-pre}}(\mathcal{A}). \end{aligned} \quad (7)$$

Similarly, one can define *distribution-oriented second preimage resistance*: given a distribution $\mathbf{d}$ over $\{0,1\}^*$, the advantage of $\mathcal{A}$ against the $\mathbf{d}$ -oriented second preimage security of $\mathbf{h}$ is

$$\mathbf{Adv}_{\mathbf{h}}^{\mathbf{d}\text{-spr}}(\mathcal{A}) := \sum_{M \in \{0,1\}^*} \Pr[M^* \xleftarrow{\$} \mathbf{d} : M^* = M] \cdot \mathbf{Adv}_{\mathbf{h}}^{M\text{-spr}}(\mathcal{A}). \quad (8)$$

Indifferentiability of XOFs. Regarding indifferentiability, we will view $\text{Sponge-}\mathbf{F}^{\mathbf{P}, \text{pd}10^*}$ as a (sponge-like) extendable-output function (XOF). According to [BDPVA11, Lef23] and NIST FIPS 202 [D⁺15], such a function $\text{XOF}(M, \nu)$ takes a message $M \in \{0,1\}^*$ and an integer ν as inputs, maps M to a long string and outputs the first ν bits of this string. We follow [BDPVA11, D⁺15, Lef23] and take such a XOF as the ideal reference of our construction. Following [BDPVA11, Lef23], we consider a variable-output-length random oracle $\mathbf{VRO} : \{0,1\}^* \mapsto \{0,1\}^*$, and consider that the random oracle queried with input-pair $(M, \nu) \in \{0,1\}^* \times \{\nu_{\min}, \dots, \nu_{\max}\}$ gives $\text{left}_{\nu}(\mathbf{VRO}(M))$. Let $\mathcal{S}^{\mathbf{VRO}}$ be a simulator that queries $\mathbf{VRO}$ and provides the same interfaces as Π and Π^{-1} . Then, for any distinguisher $\mathcal{D}$, the indifferentiability advantage of $\mathcal{D}$ against XOF^{Π} is

$$\mathbf{Adv}_{\text{XOF}^{\Pi}, \Pi, \mathcal{S}}^{\text{indiff}}(\mathcal{D}) := \left| \Pr[\mathcal{D}^{\Pi, \Pi^{-1}, \text{XOF}^{\Pi}} \Rightarrow 1] - \Pr[\mathcal{D}^{\mathcal{S}^{\mathbf{VRO}}, \mathbf{P}, \mathcal{S}^{\mathbf{VRO}}, \mathbf{P}^{-1}, \mathbf{VRO}} \Rightarrow 1] \right|.$$

Adversarial resource. Hash collision and (second) preimage adversaries would output one or two messages. In reduction proofs, the length of the output message(s) typically affects the reduction complexity and thus concrete security, and should be reflected in adversarial resources. We thus call an oracle-aided adversary $\mathcal{A}^{\mathbf{P}, \mathbf{P}^{-1}}(q, t, s, \ell_{\max})$ -*bounded*, if: (i) it makes q queries to $\mathbf{P}$ and $\mathbf{P}^{-1}$ in total, consumes t computations and s memory,

and (ii) after being padded, every message M output by $\mathcal{A}$ has at most $\ell_{\max}$ blocks. In the plain model (i.e., oracle-free setting), we use $(t, s, \ell_{\max})$ -bounded instead. Clearly, $\ell_{\max} = O(t)$, but there may be more stringent restrictions on $\ell_{\max}$ in practice: for example, SHA-256 and SHA-512 have $\ell_{\max} \leq 2^{55} + 1$ and $\ell_{\max} \leq 2^{118} + 1$ respectively [Nat15].

4.1.2 Collision and (Second) Preimage Security Bounds in the Random Permutation Model

In [GHJ+25, Sect. 4.3], the classical security bounds of $\text{Sponge-F}_h^{\Pi, \text{pd}10^*}$ in the random permutation model are given as the following theorem.

Theorem 1. *Assume $b \geq 3$. Consider the construction $\text{Sponge-F}_h^{\Pi, \text{pd}10^*}$ using a b -bit random permutation Π , output length h , capacity $c \geq h$ and rate r . Then: for any $(q, t, s, \ell_{\max})$ -bounded classical adversary $\mathcal{A}^{\Pi, \Pi^{-1}}$ with $q + 2\ell_{\max} \leq 2^b/2$, $\text{Sponge-F}_h^{\Pi, \text{pd}10^*}$ has collision, everywhere preimage and everywhere second preimage security bounds as follows:*

$$\text{Adv}_{\text{Sponge-F}_h^{\Pi, \text{pd}10^*}}^{\text{coll}}(\mathcal{A}) \leq \frac{(q + 2\ell_{\max})^2}{2^h} + \frac{2(q + 2\ell_{\max})}{2^h}; \quad (9)$$

$$\text{Adv}_{\text{Sponge-F}_h^{\Pi, \text{pd}10^*}}^{\text{epre}}(\mathcal{A}) \leq \frac{2(q + \ell_{\max})}{2^h}; \quad (10)$$

$$\text{Adv}_{\text{Sponge-F}_h^{\Pi, \text{pd}10^*}}^{\text{espr}[bl]}(\mathcal{A}) \leq \frac{\ell^2}{2^h} + \frac{2\ell}{2^c} + \frac{2\ell(q + 2\ell_{\max})}{2^c} + \frac{2(q + 2\ell_{\max})}{2^h}. \quad (11)$$

where $\ell = \max_{M \in \{0,1\}^{bl}} \{|\text{pd}10^*(M)|/r\}$ is the maximal number of blocks in padded challenge messages.

For any $(q, t, s, \ell_{\max})$ -bounded quantum adversary $\mathcal{A}^{\Pi, \Pi^{-1}}$, $\text{Sponge-F}_h^{\Pi, \text{pd}10^*}$ has quantum collision, quantum everywhere preimage and quantum everywhere second preimage security bounds as follows:

$$\text{Adv}_{\text{Sponge-F}_h^{\Pi, \text{pd}10^*}}^{\text{q-coll}}(\mathcal{A}) \leq \frac{4(8q + 16\ell_{\max} + 1)^4}{2^h} + \frac{2(8q + 16\ell_{\max} + 1)^2}{2^c}; \quad (12)$$

$$\text{Adv}_{\text{Sponge-F}_h^{\Pi, \text{pd}10^*}}^{\text{q-epre}}(\mathcal{A}) \leq \frac{2(8q + 8\ell_{\max} + 1)^2}{2^h}; \quad (13)$$

$$\text{Adv}_{\text{Sponge-F}_h^{\Pi, \text{pd}10^*}}^{\text{q-espr}[bl]}(\mathcal{A}) \leq \frac{\ell^2}{2^h} + \frac{2\ell}{2^c} + \frac{2\ell(8q + 16\ell_{\max} + 1)^2}{2^c} + \frac{2(8q + 16\ell_{\max} + 1)^2}{2^h}. \quad (14)$$

By these, when $h \leq c - \log_2 \ell_{\max}$, the concrete security bounds achieved by $\text{Sponge-F}_h^{\Pi, \text{pd}10^*}$ are listed as follows:

1. Classical collision security: $h/2$ bits;
2. Classical preimage security: h bits;
3. Classical second preimage security: h bits;
4. Quantum collision security: $h/4$ bits;
5. Quantum preimage security: $h/2$ bits;
6. Quantum second preimage security: $h/2$ bits.

The bounds are tight, except for the $h/4$ bit quantum collision security bound.

4.1.3 Indifferentiability Security Bounds in the Random Permutation Model

Following Bertoni et al. [BDPV08], we measure the cost of a construction query based on the number of permutation evaluations required in the real world to produce the output. Concretely, each call to $\text{Sponge-F}^{\Pi, \text{pd}10^*}(M, \nu)$ with ℓ -block $\overline{M}$ has a cost of ℓ . Therefore, if the adversary makes p queries to Π and Π^{-1} and q queries to $\text{Sponge-F}^{\Pi, \text{pd}10^*}$ and the padded messages have at most σ blocks in total (under our parsing role), then the *total oracle query cost* is at most $Q \leq p + \sigma$, which is the Q value introduced in Theorem 2.

With the above, in [GHJ⁺25, Sect. 6], the indifferentiability bound of $\text{Sponge-F}^{\Pi, \text{pd}10^*}$ in the random permutation model is given as the following theorem.

Theorem 2. *Assume $c \geq 3$ and $Q \geq 8$. Then, there exists a simulator $\mathcal{S}^{\text{VRO}}$ such that the following holds for any differentiator $\mathcal{D}$ with total number of oracle query costs Q :*

$$\text{Adv}_{\text{Sponge-F}^{\Pi, \text{pd}10^*}, \Pi, \mathcal{S}}^{\text{indiff}}(\mathcal{D}) \leq \frac{1}{2^c} + \frac{(4c+2)Q^2}{2^b} + \frac{12Q+14Q^2}{2^c} + \frac{5Q \log_2 Q}{2^{b-c}}.$$

Moreover, $\mathcal{S}^{\text{VRO}}$ makes at most Q queries to VRO .

Roughly speaking, indifferentiability implies that $\text{Sponge-F}^{\Pi, \text{pd}10^*}$ satisfies all security properties that can be defined using single-stage games [RSS11, DGHM13] and can be achieved by the VOL random oracle VRO , up to the indifferentiability bound of $\text{Sponge-F}^{\Pi, \text{pd}10^*}$. This includes the well-known security against multi-collision attacks and length-extension attacks. But the concrete security may be limited by $\text{Adv}_{\text{Sponge-F}^{\Pi, \text{pd}10^*}, \Pi, \mathcal{S}}^{\text{indiff}}(\mathcal{D})$. In detail, according to the concrete version of the indifferentiability composition theorem [RSS11], for any (q, t, s) -bounded classical collision adversary $\mathcal{A}^{\Pi, \Pi^{-1}}$ against $\text{Sponge-F}^{\Pi, \text{pd}10^*}$, there exists a collision adversary $\mathcal{B}^{\text{VRO}}$ against VRO , such that

$$\begin{aligned} & \Pr[\mathcal{A}^{\Pi, \Pi^{-1}}(\nu) \Rightarrow (M_1, M_2) : \text{Sponge-F}^{\Pi, \text{pd}10^*}(M_1, \nu) = \text{Sponge-F}^{\Pi, \text{pd}10^*}(M_2, \nu)] \\ & \leq \text{Adv}_{\text{Sponge-F}^{\Pi, \text{pd}10^*}, \Pi, \mathcal{S}}^{\text{indiff}}(\mathcal{A}) \\ & \quad + \Pr[\mathcal{B}^{\text{VRO}}(\nu) \Rightarrow (M_1, M_2) : \text{VRO}(M_1, \nu) = \text{VRO}(M_2, \nu)]. \end{aligned}$$

The adversary $\mathcal{B}^{\text{VRO}}$ is constructed by “combining” $\mathcal{A}^{\Pi, \Pi^{-1}}$ with the simulator $\mathcal{S}$. Therefore, $\mathcal{B}^{\text{VRO}}$ makes at most q queries to $\mathcal{RO}$, and this means the probability that $\mathcal{B}^{\text{VRO}}$ finds distinct messages M_1, M_2 with $\text{VRO}(M_1, \nu) = \text{VRO}(M_2, \nu)$ is $q^2/2^\nu$. By these,

$$\begin{aligned} & \Pr[\mathcal{A}^{\Pi, \Pi^{-1}}(\nu) \Rightarrow (M_1, M_2) : \text{Sponge-F}^{\Pi, \text{pd}10^*}(M_1, \nu) = \text{Sponge-F}^{\Pi, \text{pd}10^*}(M_2, \nu)] \\ & \leq \frac{q^2}{2^\nu} + \frac{1}{2^c} + \frac{(4c+2)Q^2}{2^b} + \frac{12Q+14Q^2}{2^c} + \frac{5Q \log_2 Q}{2^{b-c}}. \end{aligned}$$

When $\nu < c$ and $b - c \geq c/2$, it indicates collision security up to $2^{\nu/2}$ queries; when $\nu \geq c$ and $b - c \geq c/2$, it indicates collision security up to $2^{c/2}$ queries. The latter is comparable with that given in Eq. (9). Note that when $\nu \geq c$, collision security does not increase with ν .

Similarly, for any $(q, t, s, \ell_{\max})$ -bounded classical preimage adversary $\mathcal{A}^{\Pi, \Pi^{-1}}$ against $\text{Sponge-F}^{\Pi, \text{pd}10^*}$, there exists a collision adversary $\mathcal{B}^{\text{VRO}}$ against VRO , such that

$$\begin{aligned} & \Pr[\mathcal{A}^{\Pi, \Pi^{-1}}(Z, \nu) \Rightarrow M : \text{Sponge-F}^{\Pi, \text{pd}10^*}(M, \nu) = Z] \\ & \leq \text{Adv}_{\text{Sponge-F}^{\Pi, \text{pd}10^*}, \Pi, \mathcal{S}}^{\text{indiff}}(\mathcal{A}) + \Pr[\mathcal{B}^{\text{VRO}} \Rightarrow M : \text{VRO}(M, \nu) = Z] \\ & \leq \frac{q}{2^\nu} + \frac{1}{2^c} + \frac{(4c+2)Q^2}{2^b} + \frac{12Q+14Q^2}{2^c} + \frac{5Q \log_2 Q}{2^{b-c}}. \end{aligned}$$

When $\nu < c/2$ and $b - c \geq c/2$, it indicates preimage security up to 2^ν queries; when $\nu \geq c/2$ and $b - c \geq c/2$, it indicates preimage security up to $2^{c/2}$ queries. The latter is

weaker than that given in Eq. (10), since the indistinguishability security of $\text{Sponge-F}^{\Pi, \text{pd}10^*}$ is limited by roughly $c/2$ bits: this resembles the indistinguishable Merkle-Damgård variants [CDMP05, KK06].

Concrete multi-collision security bounds can be derived similarly, but for most parameters, $\text{Sponge-F}^{\Pi, \text{pd}10^*}$ only ensures $c/2$ -bit multi-collision security. This also resembles the indistinguishable Merkle-Damgård variants [Jou04].

4.2 Security analysis of the permutation

We analyze $\text{Dragon-}p$ as a keyless 1600-bit permutation. Each round is $R_r = \phi \circ \iota_r$, in which ι_r only adds a round constant to the lane $S[0][0]$. The cryptographic strength therefore comes from the diffusion layer ϕ , which is built entirely from the UPDATE transformation, and the only nonlinear operations of the permutation are the modular additions inside UPDATE. We first study UPDATE in isolation, and then bound the full-diffusion, differential, linear, differential-linear, and algebraic-degree behavior of the full permutation.

4.2.1 Properties of the Update transformation

The UPDATE transformation factors as $\text{UPDATE} = S_2 \circ L \circ S_1 \circ R$ (Figure 7), where $R(a, b, c, d, e) = (a, b, c \lll r_c, d \lll r_d, e)$, $L(a, b, c, d, e) = (a \lll r_a, (b \lll r_b) \oplus e, c, d, e \lll r_e)$, and S_1, S_2 are the only nonlinear components. Each S_i is a rotation-free S-function. S-functions were introduced by Mouha et al. [MVDP11]. Building on the geometric approach of Beyne [Bey21], Niu et al. [NBHW26] give a uniform formula for any cryptanalytic property of a Mealy machine—differential, linear, differential-linear, integral, boomerang—and we apply it to obtain the exact transition matrices of S_1 and S_2 in the differential-linear analysis below. An S-function operates on the five 64-bit words bit-slice by bit-slice, with a small carry state propagating from LSB to MSB.

Let $x_z = (a_z, b_z, c_z, d_z, e_z) \in \mathbb{F}_2^5$ be the 5-bit chunk at bit position z , with a_z the most significant bit. An S-function is a Mealy machine $F : \mathbb{F}_2^5 \times \mathbb{F}_2^\sigma \rightarrow \mathbb{F}_2^5 \times \mathbb{F}_2^\sigma$ that maps $(x_z, s_z) \mapsto (y_z, s_{z+1})$. Processing runs from $z = 0$ (LSB) to $z = 63$ (MSB); the initial carry is $s_0 = (0, \dots, 0)$. The majority function is $\text{maj}(u, v, w) = uv \oplus uw \oplus vw$.

The S-function S_1 carries a 3-bit state $s_z = (\kappa_1, \kappa_2, \kappa_3)$:

$$\begin{aligned} b'_z &= b_z \oplus e_z \oplus \kappa_1, & \kappa'_1 &= \text{maj}(b_z, e_z, \kappa_1), \\ d'_z &= d_z \oplus b'_z, \\ a'_z &= a_z \oplus d'_z \oplus \kappa_2, & \kappa'_2 &= \text{maj}(a_z, d'_z, \kappa_2), \\ c'_z &= c_z \oplus a'_z, \\ e'_z &= e_z \oplus c'_z \oplus \kappa_3, & \kappa'_3 &= \text{maj}(e_z, c'_z, \kappa_3). \end{aligned}$$

The next carry is $s_{z+1} = (\kappa'_1, \kappa'_2, \kappa'_3)$. At the word level this is $b \leftarrow b \boxplus e$, $d \leftarrow d \oplus b$, $a \leftarrow a \boxplus d$, $c \leftarrow c \oplus a$, $e \leftarrow e \boxplus c$.

The S-function S_2 carries a 2-bit state $s_z = (\kappa_1, \kappa_2)$:

$$\begin{aligned} d'_z &= d_z \oplus b_z \oplus \kappa_1, & \kappa'_1 &= \text{maj}(d_z, b_z, \kappa_1), \\ a'_z &= a_z \oplus d'_z, \\ c'_z &= c_z \oplus a'_z \oplus \kappa_2, & \kappa'_2 &= \text{maj}(c_z, a'_z, \kappa_2), \\ e'_z &= e_z \oplus c'_z, \\ b'_z &= b_z. \end{aligned}$$

The next carry is $s_{z+1} = (\kappa'_1, \kappa'_2)$. Lane b passes through unchanged. At the word level: $d \leftarrow d \boxplus b$, $a \leftarrow a \oplus d$, $c \leftarrow c \boxplus a$, $e \leftarrow e \oplus c$.

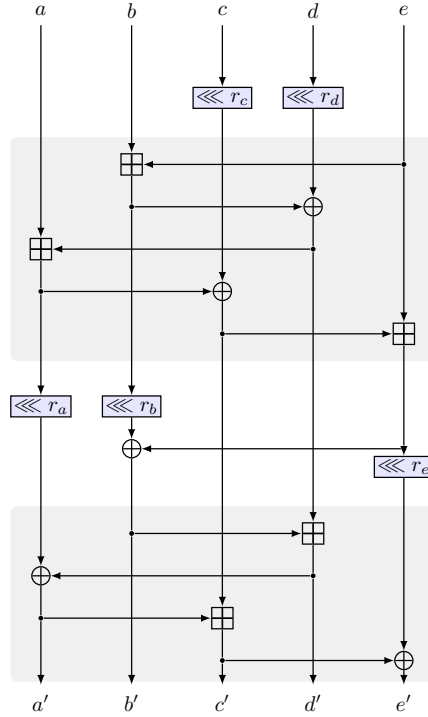


Figure 7: The UPDATE transformation of Fig. 4 redrawn with its rotations grouped, giving the factorization $\text{UPDATE} = S_2 \circ L \circ S_1 \circ R$. A rotation commutes with XOR but not with modular addition, so r_c and r_d , which act on the still untouched words c and d , move to the front and form the rotation layer R . The first rotation-free block $b \boxplus e$, $d \oplus b$, $a \boxplus d$, $c \oplus a$, $e \boxplus c$ is the S-function S_1 . The rotations r_a, r_b, r_e together with the cross XOR $b \oplus e$ form the linear layer L . The remaining rotation-free block $d \boxplus b$, $a \oplus d$, $c \boxplus a$, $e \oplus c$ is the S-function S_2 .

This explicit S-function model of S_1 and S_2 is what drives the differential-linear analysis in Section 4.2: the method of Niu et al. [NBHW26] turns each S_i into exact transition matrices, which the round-based differential-linear approximation propagates layer by layer, and which also yield the integral and boomerang transitions of the full UPDATE.

4.2.2 Full diffusion

To evaluate the diffusion capability of the Dragon- p , we performed deterministic bit-level dependency tracking. Each state bit is associated with the set of input bits on which it depends. Initially, every bit depends only on itself. Dependencies are then propagated through the round function by modeling XOR operations as set unions, rotations as bit permutations, and modular additions through carry propagation. For the inverse permutation, modular subtractions are handled analogously through borrow propagation. Since round-constant injections introduce no state dependencies, they are omitted from the analysis.

A state bit is considered fully diffused once it depends on all 1600 input state bits. The numbers of fully diffused output bits after each round in the forward and inverse directions are summarized in Table 7.

Table 7: Bit-level dependency diffusion of the Dragon- p .

Direction	Round 1	Round 2	Round 3	Full diffusion
Forward	568/1600	1600/1600	1600/1600	2 rounds
Inverse	0/1600	879/1600	1600/1600	3 rounds

The above results show that full diffusion is achieved after two rounds in the forward direction and three rounds in the inverse direction. The dependency-tracking results are independently validated by random bit-flip experiments, which yielded identical full-diffusion rounds. Relative to the 16-round permutation, this demonstrates rapid dependency propagation and a substantial diffusion margin.

4.2.3 Differential and linear cryptanalysis

Search model. We evaluate the resistance of Dragon- p against XOR-differential and linear cryptanalysis by searching for low-weight single trails of the round-reduced permutation. The search is based on the MILP framework of Fu *et al.* [FWG⁺16], originally developed for Speck and adapted here to the full Dragon- p round function. Modular addition is the only nonlinear operation and is therefore modeled explicitly, whereas rotations and XORs are propagated exactly over $\mathbb{F}_2$.

For the differential search, modular additions are constrained by the exact XOR-differential condition of Lipmaa and Moriai [LM02]. For the linear search, they are modeled using Wallén’s linear-mask automaton [Wal03]. The objective function is the conventional trail weight, namely $-\log_2 p$ for a differential trail of probability p , and $-\log_2 |C|$ for a linear trail of correlation C . Since round constants do not affect either probabilities or correlations, they are omitted from the weight computation.

The resulting constraints are translated into STP bit-vector format and solved with CryptoMiniSat through STP. For a given number of rounds, the solver is used either to determine a minimum-weight trail or to test whether a trail above a prescribed probability or correlation threshold exists. In the latter case, an UNSAT result certifies the absence of such a high-probability or high-correlation trail within the exact model. Consequently, the results reported below distinguish concrete trails obtained by the solver from bounds certified through UNSAT proofs. Throughout this section, only single-trail characteris-

tics are considered and effects arising from differential clustering or linear hulls are not included.

Differential cryptanalysis. For the XOR-differential search, the best one-round characteristic has probability 2^{-2} . For two rounds, the best trail returned by the solver has probability 2^{-45} , and the STP instance with target probability 2^{-32} is UNSAT.

For three and four rounds, direct exact search becomes expensive because of the large number of modular additions and intermediate bit-vector variables. We therefore employ a trail-concatenation strategy, constructing compatible $(2 + 1)$ - and $(2 + 2)$ -round trails for the three- and four-round searches, respectively. The resulting best trails have probabilities 2^{-813} and 2^{-2444} .

Linear cryptanalysis. For the linear search, the best one-round trail has absolute correlation 2^{-1} . For two rounds, the best trail has absolute correlation 2^{-24} , while the STP instance corresponding to a target absolute correlation of 2^{-20} is UNSAT.

For three and four rounds, we employ the same trail-concatenation strategy as in the differential search. The resulting best trails have absolute correlations 2^{-267} and 2^{-1524} , respectively.

Summary. The results of the differential and linear trail searches are summarized in Table 8. The UNSAT entries provide certified bounds within the exact model. In particular, no two-round differential trail of *Dragon-p* has probability at least 2^{-32} , and no two-round linear trail has absolute correlation at least 2^{-20} . Equivalently, every two-round differential trail captured by the model has differential weight greater than 32, and every two-round linear trail has correlation weight greater than 20.

For both differential and linear cryptanalysis, the substantial increase in trail weight from one to two rounds indicates that high-probability differential characteristics and high-correlation linear approximations are rapidly suppressed. Since the three- and four-round searches rely on trail concatenation, the reported probabilities and correlations should not be interpreted as proven optimal values and do not exclude the existence of better trails. Nevertheless, the consistent growth of trail weights across the analyzed rounds suggests that high-probability characteristics and high-correlation approximations become increasingly unlikely as the number of rounds increases. Considering that the full *Dragon-p* permutation employs 16 rounds, the current results indicate a substantial security margin against single-trail differential and linear attacks.

The detailed round-by-round trails are provided in Appendix A, Tables 17–20.

Table 8: Summary of differential and linear trail searches for reduced-round *Dragon-p*.

Attack Type	Rounds	Best trail	Certified bound
Differential	1	2^{-2}	$p \leq 2^{-2}$
	2	2^{-45}	$p < 2^{-32}$
	3	2^{-813}	–
	4	2^{-2444}	–
Linear	1	2^{-1}	$ C \leq 2^{-1}$
	2	2^{-24}	$ C < 2^{-20}$
	3	2^{-267}	$ C < 2^{-216*}$
	4	2^{-1524}	–

* This bound fixes the optimal two-round linear trail (absolute correlation 2^{-24}) and bounds, by exact search, the correlation of the third round appended to it; it therefore bounds only the three-round trails that extend this two-round trail.

4.2.4 Differential-linear cryptanalysis

We bound the best (higher-order) differential-linear distinguisher of **Dragon- p** with the round-based approximation of the differential-linear correlation of Hu et al. [HNW26]. The method evaluates the differential-linear correlation

$$\text{Cor}[\Delta \rightarrow \Gamma] = 2^{-n} \sum_x (-1)^{\Gamma \cdot (F(x) \oplus F(x \oplus \Delta))}$$

as one coordinate of the correlation vector of the 2-wise transform $F^{\times 2}$ in the geometric approach. This vector is propagated layer by layer and the coordinate indexed by the output mask Γ is read off at the end, which avoids the assumption of independence between the differential and the linear part that classical differential-linear analysis makes at the switch. The propagation requires the permutation to be written as an alternation of a nonlinear layer S , built from parallel building blocks, and an $\mathbb{F}_2$ -linear layer L . It rests on two assumptions: a rank-one assumption, that the correlation vector factors as a tensor product over the parallel blocks, which is where the approximation error lives; and an optional zero value-mask assumption, which is exact for a keyed primitive averaged over its round keys and a well-supported heuristic for a keyless permutation such as **Dragon- p** . The method extends to higher-order differential-linear correlations by replacing the 2-wise transform $F^{\times 2}$ with the d -wise transform $F^{\times d}$.

Configuration for **Dragon- p .** We cast the 16-round permutation in the required alternating form through the factorization $\text{UPDATE} = S_2 \circ L \circ S_1 \circ R$ of Section 4.2 (Figure 7). The nonlinear layers are the S-functions S_1 and S_2 : in the diagonal stage they act on the five diagonals in parallel, and in the column stage on the five columns in parallel, so each stage contributes one parallel layer of S_1 followed by one of S_2 . Everything else is $\mathbb{F}_2$ -linear and is absorbed into the linear layer: the rotation layer R , the linear layer L inside UPDATE , the diagonal-to-column lane wiring of ϕ , and the round-constant injection ι_r . The constant of ι_r acts only on the value mask. In the full 2-wise path it is retained as a sign on the value-mask coordinates, and it vanishes only under the zero value-mask assumption. Because S_1 and S_2 are S-functions rather than memoryless S-boxes, their 2-wise transition matrices are not S-box lookup tables but are obtained from the Mealy-machine transition matrices of Section 4.2; these matrices replace the S-box correlation tables used in the original formulation of the method, and the carry state is handled exactly by the geometric-approach formula.

Round-based propagation. We retain the full 2-wise correlation vector across rounds and apply the rank-one factorization only at the input of each S-function. The value mask is fixed to zero only on the input difference and on the read-off output mask, where this is exact, so the only approximation is the rank-one factorization. At each factorization point the correlation vector is a 5×64 array over 32×32 value/difference masks: the five parallel UPDATE s, 64 slices per UPDATE , and a 5-bit value mask together with a 5-bit difference mask on each slice. One round acts as

$$\sigma^{(r)} \xrightarrow{L_1 \circ S_1} \gamma^{(r)} \xrightarrow{L_2 \circ S_2} \theta^{(r)} \xrightarrow{L_1 \circ S_1} \xi^{(r)} \xrightarrow{L_3 \circ S_2} \sigma^{(r+1)}, \quad (15)$$

where the first two maps form the diagonal stage and the last two the column stage, and $\sigma^{(0)}$ is obtained from the input difference Δ pushed through the initial rotation R . The three linear layers are L_1 , the layer L inside UPDATE ; L_2 , the diagonal-to-column lane wiring followed by the rotation R of the column UPDATE ; and L_3 , the column-to-diagonal lane wiring followed by the rotation R of the next diagonal UPDATE .

Each $L \circ S$ map is evaluated jointly: $L \circ S$ is kept exact and the factorization is deferred to the next S-function input. For an output mask γ' supported on a single output block, we pull it back through $L^\top$ to the S-output mask $\gamma = L^\top \gamma'$, which is fixed and precomputed once per linear layer. Because the five UPDATE s of a stage are independent, γ splits over

the source UPDATES and the correlation factors as the product of the five per-UPDATE correlations, each evaluated exactly along the 64-slice carry chain of its S-function with the Mealy transition matrices of Section 4.2.

Results. Table 9 reports, for one and two rounds, the single-bit pair (Δ, Γ) of largest round-based correlation, together with a Monte-Carlo measurement of the same pair. A state bit is written $S[x][y]_z$. After one round the round-based correlation is exact and matches the measurement; one round still admits deterministic single-bit relations. After two rounds the best single-bit correlation is $2^{-0.74}$ against a measured $2^{-0.42}$, so the round-based estimate has the correct order of magnitude while underestimating the measured correlation; it is therefore optimistic from a security standpoint.

Table 9: Best single-bit differential-linear distinguisher of **Dragon- p** by round count: round-based estimate (RB) and Monte-Carlo measurement (MC) of the same (Δ, Γ) . A state bit is denoted $S[x][y]_z$. The MC columns use 2^{20} (1 round) and 2^{24} (2 rounds) random inputs. For $r \geq 3$ neither the round-based estimate nor a direct Monte-Carlo measurement over the whole permutation resolves the correlation; it is instead estimated by the three-part construction described in the text.

Rounds	Δ	Γ	RB correlation	MC correlation
1	$S[3][0]_{27}$	$S[2][1]_0$	1	1.000
2	$S[2][0]_{39}$	$S[1][3]_0$	$2^{-0.74}$	$2^{-0.42}$

Three-part construction. Beyond two rounds the round-based approximation no longer resolves the differential-linear correlation, and a direct Monte-Carlo measurement over the whole permutation falls below the experimental detection threshold. We therefore estimate the differential-linear correlation by the standard three-part decomposition [LH94, BDKW19]. We write the round-reduced permutation as

$$E = E_\ell \circ E_m \circ E_d,$$

where E_d covers the first r_d rounds, E_m the middle r_m rounds, and E_ℓ the last r_ℓ rounds. The part E_d is treated differentially: an input difference Δ propagates to an intermediate difference Δ_m at the input of E_m with probability p . The part E_ℓ is treated linearly: an output mask Γ pulls back to an intermediate mask Γ_m at the output of E_m with correlation q . The middle part E_m is evaluated experimentally: its differential-linear correlation

$$r = \text{Cor}_x[\Gamma_m \cdot (E_m(x) \oplus E_m(x \oplus \Delta_m))]$$

is measured directly by Monte-Carlo, which keeps the dependency across the two switches exact and avoids the independence assumption made there by a closed-form analysis. Assuming the three parts are independent, the differential-linear correlation of E is estimated as

$$\text{Cor}[\Delta \rightarrow \Gamma] \approx p \cdot r \cdot q^2.$$

4.2.5 Integral cryptanalysis

We evaluate integral distinguishers of the **Dragon- p** using monomial prediction [HSWW20]. The only nonlinear operation of **Dragon- p** is the modular addition. To obtain a precise model for modular addition, we adopt the perfect MP model for modular addition [BS05, HY24]. Our search follows the MILP framework of [HY24]. However, for a better trade-off of security and efficiency, we test only the existence of the monomial trails rather than enumerating all the trails. For the integral search, taking the rate part as the active set and fixing the remaining state bits, the search finds no integral distinguisher for the

15-round *Dragon-p*. We do not explicitly consider weak distinguishers such as zero-sum distinguishers, as they do not provide any threat for the security of *Dragon-p*.

4.2.6 Impossible differential and zero-correlation cryptanalysis

As stated above, the *Dragon-p* permutation achieves full diffusion after two rounds in the forward direction and three rounds in the inverse direction. Consequently, diffusion-based miss-in-the-middle impossible differential and zero-correlation distinguishers are expected to be around

$$2 + 3 = 5$$

rounds.

4.3 Security analysis of the hash function

The mode-level proofs earlier in this section give generic collision, (second) preimage, and indistinguishability bounds under the assumption that *P* is ideal, and the analysis of Section 4.2 shows that *Dragon-p* has no exploitable structural weakness. Here we analyze the concrete hash functions *Dragon-512*, *Dragon-768*, *Dragon-1024*, and *Dragon-XOF* against dedicated attacks, report the classical and quantum security strengths required by the call, and address the additional security features.

4.3.1 Preimage and second-preimage resistance

Differential-linear preimage. The differential-linear preimage framework of Niu et al. [NHS⁺24] speeds up a preimage search by exploiting highly biased differential-linear approximations (δ, λ) of the target one-way function: an input difference δ in the message together with an output mask λ on the digest for which $\lambda \cdot (F(x) \oplus F(x \oplus \delta))$ is strongly biased lets the search reject message candidates without evaluating the hash. The attainable speed-up is governed entirely by the bias of these approximations, and the framework is effective only when highly biased differential-linear distinguishers exist. For *Dragon-p* no such distinguisher survives beyond a few rounds. As reported in Section 4.2 (Table 9), the best single-bit differential-linear correlation equals 1 after one round and is already only about $2^{-0.4}$ after two rounds. Beyond two rounds the round-based approximation no longer resolves the correlation and a direct Monte-Carlo measurement over the whole permutation falls below the experimental detection threshold; the correlation must instead be estimated by the three-part decomposition $E = E_\ell \circ E_m \circ E_d$ of Section 4.2, whose product form $p \cdot r \cdot q^2$ decays rapidly with the round count. The strongly biased approximations that the framework requires therefore do not survive more than a few rounds. Even granting that two or three rounds remain exploitable, the full 16-round *Dragon-p* retains a margin of at least 13 rounds, so the framework of [NHS⁺24] cannot be instantiated against the full permutation. We conclude that differential-linear preimage attacks do not threaten the *Dragon* hash family, whose preimage resistance stays at the generic level established above.

4.3.2 Collision resistance

Search model. To evaluate the collision resistance of the *Dragon* hash function, we performed single-block collision searches under two collision search models: the standard model and the feed-forward model. Both models consider a single message absorption followed by one invocation of the underlying *Dragon-p* permutation and final hash extraction.

In the standard model, two internal states differ only in the absorbed message block. Thus, differences are allowed only in the rate part, while the capacity difference is fixed to zero. A collision is obtained by requiring the final hash difference to be zero.

In the feed-forward model, differences are additionally allowed in the capacity part. Owing to the feed-forward operation, the output difference of the underlying *Dragon- p* over the digest bits is constrained to equal the corresponding capacity difference, so that the two differences cancel after feed-forward, yielding identical hash outputs.

Both collision search models are built upon the exact differential model described in Section 4.2.3, with additional constraints imposed on the initial and final state differences.

Collision search results. We performed collision searches for one and two rounds under both the standard and feed-forward collision models.

For one round, our search found collision trails with probabilities 2^{-72} , 2^{-196} and 2^{-768} under the standard model for *Dragon-512*, *Dragon-768* and *Dragon-1024*, respectively. Under the feed-forward model, the best one-round collision trails found by our search have probabilities 2^{-22} , 2^{-68} , and 2^{-360} for *Dragon-512*, *Dragon-768* and *Dragon-1024*, respectively.

For two rounds, no collision trail with probability exceeding the birthday bound was found for any *Dragon* instance under either collision model.

Summary. The collision search results are summarized in Table 10. For all three *Dragon* variants, no two-round collision trail with probability exceeding the birthday bound was found under either collision model. Consequently, the current results provide no evidence of collision attacks outperforming the generic birthday attack after two rounds.

The reported one-round trails correspond to the best results found within the available search budget and do not exclude the existence of better collision trails. Nevertheless, the current results indicate that the full *Dragon* hash function retains a comfortable security margin against differential collision attacks.

Table 10: Best one-round collision trails found under two collision search models.

Version	Standard model	Feed-forward model
<i>Dragon-512</i>	2^{-72}	$2^{-22\dagger}$
<i>Dragon-768</i>	2^{-196}	2^{-68}
<i>Dragon-1024</i>	2^{-768}	2^{-360}

$\dagger$ indicates a proven optimal trail.

4.3.3 Quantum security

(Second) preimage attacks. In the quantum random-permutation setting used above, Eq. (13) gives $h/2$ -bit generic preimage security for the fixed-output hash. In the preimage and second-preimage discussion below, write H for the fixed-output hash instance under discussion. The matching generic attack is Grover search [Gro96]. For a candidate message family S and a target digest O , the marked messages are those satisfying $H(M) = O$. Grover search starts from the uniform superposition over S , applies a phase oracle for this condition, and then applies the diffusion reflection. When the marked fraction is about 2^{-h} , the Grover cost is

$$T_{\text{Grover}} \approx 2^{h/2}.$$

If a reversible preimage predicate increases the marked fraction by a factor 2^σ , after its success probability is included, the corresponding quantum cost is

$$T_{\text{Grover}}(\sigma) \approx \sqrt{2^h/2^\sigma} = 2^{(h-\sigma)/2}.$$

This conversion requires a reversible implementation of the filter and final check.

For the 16-round fixed-output **Dragon** instances, the $h/2$ bound gives generic Grover costs 2^{256} , 2^{384} , and 2^{512} for **Dragon-512**, **Dragon-768**, and **Dragon-1024**, respectively. In the **Dragon-512** case, one absorbed block already supplies at least h variable bits. In the **Dragon-768** and **Dragon-1024** cases, the generic search uses enough padded rate blocks to obtain message families of size at least 2^h . The reduced-round differential-linear conversion below is separate from these full-instance costs.

The differential-linear preimage analysis above can be converted to the quantum setting only when it supplies a reversible filter with quantified gain $2^{\sigma_{\text{DL}}}$. If such a reduced-round filter is instantiated, amplitude amplification gives

$$2^{(h-\sigma_{\text{DL}})/2}$$

evaluations of the reduced-round permutation, plus the cost of the reversible filter. For **Dragon- p** , the available differential-linear correlations do not give a useful full-instance filter. This conversion therefore remains reduced-round and does not affect the 16-round costs above.

For second preimages, Eq. (14) is the applicable quantum bound. Given a challenge message M^* , the adversary must find $M' \neq M^*$ with the same digest. For message lengths for which the ℓ - and ℓ_{max} -dependent terms in this bound are below the Grover term, the generic estimate remains $T_{2\text{nd}} \approx 2^{h/2}$.

For reduced-round second preimages, the differential-linear preimage conversion gives only a conditional attack-cost estimate. It also requires an additional disjointness condition. Given M^* , set $O^* = H(M^*)$. If there is a message family S^* with $M^* \notin S^*$ on which the same filter and final check are reversible, then any marked $M' \in S^*$ satisfying $H(M') = O^*$ is a second preimage. Under this condition, the quantum cost is $T_{\text{DL-2nd}} \approx 2^{(h-\sigma_{\text{DL}})/2}$. This is a conditional attack-cost estimate. It assumes that this message family supports the same filter, and it remains a reduced-round estimate. For the 16-round fixed-output **Dragon** instances, the attacks considered here do not lower the generic quantum preimage or second-preimage estimates. When this domination condition holds for second preimages, the expected costs for **Dragon-512**, **Dragon-768**, and **Dragon-1024** remain 2^{256} , 2^{384} , and 2^{512} , respectively.

Collision attacks. For collision search, the digest value is not fixed in advance. The relevant generic quantum attack is the BHT collision algorithm [BHT98]. It stores L hash outputs, then uses Grover search to find a fresh message whose digest hits this stored set. Balancing the table size with the Grover search cost gives

$$T_{\text{BHT}} = \Theta(2^{h/3}), \quad S_{\text{BHT}} = \Theta(2^{h/3}).$$

Later quantum collision-search analyses use the same time-memory tradeoff as the generic comparison point [CNS17]. For **Dragon-512**, **Dragon-768**, and **Dragon-1024**, the generic quantum collision exponents are $512/3$, $768/3$, and $1024/3$, respectively.

For the dedicated comparison, we use the two-sided extension idea from classical reduced-permutation collision searches [DGPW12, MRST09] and count only the two probabilistic extensions. This gives an estimate favorable to the attacker, since internal matching, message reachability, and final-digest constraints are assigned zero cost. For a u -round forward extension and a v -round backward extension, write

$$p_{\text{F},u} = 2^{-w_{\text{F},u}}, \quad p_{\text{B},v} = 2^{-w_{\text{B},v}}.$$

Amplitude amplification then gives the optimistic collision exponent

$$K_{\text{col},u+v}^{\text{q}} = \frac{1}{2}(w_{\text{F},u} + w_{\text{B},v}) = -\frac{1}{2} \log_2(p_{\text{F},u} p_{\text{B},v}).$$

Such a quantum collision search can beat BHT only if

$$K_{\text{col},u+v}^q < \frac{h}{3}.$$

Since these sponge-forward conditions are not charged in the estimate, enforcing them would not decrease the cost and may make the approach inapplicable.

Table 11 evaluates these optimistic quantum collision exponents from the available one-round and two-round extension probabilities. Following the two-sided extension restrictions used by Duc et al. [DGPW12], the probabilities $p_{F,u}$ and $p_{B,v}$ are the effective probabilities used to filter candidate pairs after the forward and backward differential extensions are embedded into the hash collision search.

Table 11: Optimistic quantum collision exponents for Dragon two-sided extensions when internal matching is not charged.

Instance	$p_{F,1}$	$p_{F,2}$	$p_{B,1}$	$p_{B,2}$	$K_{\text{col},1+1}^q$	$K_{\text{col},2+2}^q$	$K_{\text{col},2+2}^q - h/3$
Dragon-512	2^{-4}	$\geq 2^{-761}$	2^{-11}	$\geq 2^{-171}$	7.50	466.00	295.33
Dragon-768	2^{-8}	—	2^{-11}	—	9.50	—	—
Dragon-1024	2^{-27}	—	2^{-11}	—	19.00	—	—

The optimistic 1 + 1-round extension exponents lie below $h/3$ for all three parameter sets, but they account only for the two one-round extensions and do not constitute full-hash collision attacks. For Dragon-512, the boundary-value estimate for the two two-round extensions is 466.00. At the displayed boundary values, this is 295.33 above $512/3$. For Dragon-768 and Dragon-1024, the search found no two-sided extension with more rounds beyond the listed 1 + 1-round data. This comparison already assigns zero cost to internal matching, message reachability, and final-digest constraints. Thus, under this optimistic accounting, the searches did not find any extension with more rounds. From the differential-complexity estimates in Table 11, no full 16-round quantum collision attack below the BHT costs $2^{512/3}$, $2^{768/3}$, and $2^{1024/3}$ exists within the searched two-sided extension setting for Dragon-512, Dragon-768, and Dragon-1024, respectively.

4.3.4 Statistical randomness tests

We evaluated the statistical randomness of the output streams generated by Dragon-512, Dragon-768, and Dragon-1024 using version 2.1.2 of the NIST Statistical Test Suite (STS). For each variant, an output bit stream was generated by hashing 64-bit big-endian counter values with a fixed offset $M_i = \text{uint64_be}(i + 2^{34})$ and concatenating the resulting digests:

$$H(M_0) \| H(M_1) \| H(M_2) \| \cdots .$$

The resulting stream was then split into 100 sequences of length 10^6 bits, yielding a total of 10^8 tested bits for each algorithm. All STS tests were executed with significance level $\alpha = 0.01$. A test item is regarded as successful if it satisfies both the STS pass-rate criterion and the p -value uniformity criterion. For the Random Excursions (RE) and Random Excursions Variant (RV) tests, the effective sample size may be smaller than 100 because only sequences containing a sufficient number of random-walk cycles are eligible for these tests.

Table 12: Summary of NIST STS results for Dragon output streams. “Items” denotes the number of reported STS entries, “Fail” the number of failed entries, “Min. Prop.” the minimum passing proportion, “RE/RV N” the effective sample size used by the RE and RV tests, “Min. RE/RV” the minimum passing proportion among those tests, and “Min. U- p ” the minimum uniformity p -value.

Variant	Items	Fail	Min. Prop.	RE/RV N	Min. RE/RV	Min. U- p
512	188	0	97/100	69	67/69	0.002792
768	188	0	97/100	58	57/58	0.003712
1024	188	0	96/100	58	56/58	0.001757

Table 12 shows that all three variants successfully passed the STS evaluation. No failed test items were observed, all passing proportions satisfied the corresponding STS lower bounds, and all uniformity p -values exceeded 0.0001. These results provide no evidence of statistically significant deviations from randomness in the output streams generated by Dragon.

4.3.5 Length extension resistance

A length extension attack attempts to compute the hash or XOF output of an extended message $M \parallel X$, given only the output associated with an unknown message M . To mount such an attack against Dragon, an adversary must obtain a valid intermediate state from which the additional message blocks in X can be absorbed.

For the fixed-output instances Dragon-512, Dragon-768, and Dragon-1024, a single output extraction is performed. An h -bit digest therefore reveals only h bits of the final $b = 1600$ -bit state. Assuming that the final state is uniformly distributed, a given digest is consistent with approximately 2^{b-h} internal states. In particular, the numbers of candidate states are 2^{1088} , 2^{832} , and 2^{576} for Dragon-512, Dragon-768, and Dragon-1024, respectively. Hence, the digest does not determine the correct internal state required to continue message absorption.

For Dragon-XOF, a sufficiently long output may determine the squeezing state. However, knowledge of this state only allows the adversary to continue squeezing the output stream associated with M and it does not provide a valid state for absorbing additional message blocks. This follows from the domain separation applied during final-block processing. Each non-final message block is injected as

$$M[i] \parallel 0^c,$$

whereas the last padded block is injected as

$$M[\ell] \parallel C, \quad C = 0^{c-1} \parallel 1.$$

Thus, the squeezing state is obtained only after the finalization constant C has been injected. It is therefore a finalized state and cannot be reused as the non-final absorption state required for computing the output of $M \parallel X$.

Moreover, the feed-forward operation maps an input state Y to

$$P(Y) \oplus (0^r \parallel \text{right}_c(Y)).$$

Consequently, recovering the state preceding finalization cannot be accomplished by simply applying P^{-1} to the squeezing state. An adversary would have to solve the feed-forward relation and simultaneously account for the difference between the domain-separation constant C with the zero inner-part injection used for a non-final block.

Therefore, the fixed-output digests do not reveal the internal continuation state, and even a fully determined **Dragon**-XOF squeezing state is a domain-separated finalized state rather than a valid absorption state. Hence, the classical message length extension attack is not applicable to either the fixed-output or the XOF instances of **Dragon**.

5 Performance

5.1 Software performance

We evaluate the software performance of **Dragon** on two representative 64-bit software platforms. The evaluation covers the three fixed-output hash instances, **Dragon**-512, **Dragon**-768, and **Dragon**-1024, and includes SHA3-512 and SM3 as standard baselines for comparison. The primary metric is cycles per byte (cpb), computed from the measured cycle count divided by the input length. Lower cpb values indicate higher software efficiency.

Design-level software rationale. The software efficiency of **Dragon** derives from its ARX structure, which relies exclusively on modular additions, cyclic rotations, and XOR operations on 64-bit words. Every measurement reported here uses only these general-purpose ALU instructions: **Dragon** calls for no cryptographic instruction-set extension such as Intel’s AES-NI or SHA Extensions, or the Armv8 SHA-512/SHA-3 instructions. AES- and SHA-based primitives, by contrast, rely on such extensions to reach their headline speeds and slow down markedly without them. The UPDATE transformation deliberately interleaves rotations with modular additions. This scheduling strategy exploits the barrel-shifter capability of ARM processors while taking into account that rotation instructions can be issued through only two execution ports on x86 processors. Furthermore, each XOR operation immediately follows its corresponding rotation; on ARM architectures, this sequence can be issued as a single shifted-register logical instruction. As a result, the critical instruction depth of a single UPDATE transformation is only 10.

Each round of **Dragon** executes ten UPDATE transformations: five independent updates on the cyclic diagonals, followed by five independent updates on the columns. Equivalently, each round contains 50 modular additions, 50 rotations, and 50 XOR operations, in addition to the round-constant injection. The 1600-bit state consists of 25 64-bit lanes. On an ARM64 Cortex-A76-class core, arithmetic and logical instructions are supported by three execution ports, and the architectural register file provides 32 general-purpose registers. Consequently, all 25 state words can remain in registers throughout the optimized permutation. In contrast, x86 Core-class processors provide four execution ports for arithmetic and logical instructions but only 16 general-purpose registers, resulting in significant register pressure when holding all 25 state words. The optimized x86 implementation therefore schedules three UPDATE transformations in parallel, reducing register spills while preserving instruction-level parallelism. Under this scheduling model, the theoretical cost of one round is approximately 40 cycles, whereas the measured implementation requires 38.8 cycles per round, closely matching the theoretical analysis.

Benchmark setup. The ngcc benchmark was run in single-core mode on the two platforms summarized in Table 13. The test set uses the S1–S8 message sizes specified by the self-assessment guidelines, namely 32, 128, 512, 1024, 4096, 8192, 16384, and 65536 bytes. The same ngcc measurement procedure is used for the SHA3-512 baseline. The measured cpb values are reported in Tables 14 and 15.

Table 13: Software benchmark platforms.

Item	x86-64 platform	ARM platform
Processor	Intel Xeon(R) Platinum 8358 CPU @ 2.60 GHz	Rockchip RK3588 @ 2.0 GHz
Operating system	Ubuntu 22.04 LTS (kernel 6.8.0)	Ubuntu 22.04 (kernel 5.10.160)
Compiler	GCC 11.4	GCC 9.4
Build tool	CMake 3.22.1	CMake 3.16.3
Benchmark mode	Single core	Single core

Table 14: Software performance comparison of SHA3-512, SM3, and the x86-64 implementation of Dragon on Intel Xeon(R) Platinum 8358. Values are cycles per byte (cpb); lower is better.

Instance	S1 (32 B)	S2 (128 B)	S3 (512 B)	S4 (1024 B)	S5 (4096 B)	S6 (8192 B)	S7 (16384 B)	S8 (65536 B)
SM3 (256) [†]	70.06	36.74	14.43	9.28	8.59	8.51	8.47	8.45
SHA3-512	103.73	34.97	20.43	13.46	12.49	12.48	12.47	12.46
Dragon-512	22.96	11.00	6.44	5.65	5.06	4.96	4.91	4.87
Dragon-768	23.17	10.92	7.66	6.91	6.60	6.57	6.52	6.49
Dragon-1024	23.00	15.62	11.18	10.44	9.88	9.80	9.73	9.70

Table 15: Software performance comparison of SHA3-512, SM3, and the ARM implementation of Dragon on Rockchip RK3588. Values are cycles per byte (cpb); lower is better.

Instance	S1 (32 B)	S2 (128 B)	S3 (512 B)	S4 (1024 B)	S5 (4096 B)	S6 (8192 B)	S7 (16384 B)	S8 (65536 B)
SM3 (256) [‡]	94.80	20.44	11.00	9.42	8.26	8.05	7.95	7.88
SHA3-512 [†]	67.58	25.59	19.41	17.32	15.74	15.60	15.54	15.49
Dragon-512	51.24	17.27	7.63	6.04	4.84	4.64	4.54	4.47
Dragon-768	51.31	17.41	8.80	7.18	6.24	6.11	6.01	5.96
Dragon-1024	51.85	21.93	12.19	10.57	9.35	9.15	9.05	8.99

[†] The SHA3-512 baseline is a portable software implementation and does not use the Armv8.2 SHA-3 instructions (EOR3, RAX1, XAR, BCAX); x86-64 provides no SHA-3 instruction extension.

[‡] The SM3 baseline is a portable software implementation and does not use the Armv8.2 SM3 instructions (SM3SS1, SM3TT1A/1B, SM3TT2A/2B, SM3PARTW1/2); x86-64 provides no SM3 instruction extension.

Observations. The results show a consistent decrease in cpb as the input length increases. This is the normal profile of a hash implementation with non-negligible fixed costs for initialization, padding, and finalization: these costs dominate short inputs, but are amortized for longer messages. Consequently, the S1 results are close across the three Dragon instances on each platform, whereas the long-message results more clearly reflect the steady-state processing cost of each parameter set. Against SHA3-512, all three Dragon instances have lower cpb at every tested input length on both platforms. This includes not only the fastest Dragon-512 instance, but also the larger Dragon-768 and Dragon-1024 instances. For example, at S8 the x86-64 implementation reaches 4.87, 6.49, and 9.70 cpb for Dragon-512, Dragon-768, and Dragon-1024, respectively, all below the 12.46 cpb measured for SHA3-512. On ARM, the same three Dragon instances reach 4.47, 5.96, and 8.99 cpb, again all below the 15.49 cpb measured for SHA3-512. Thus, even the larger Dragon-768 and Dragon-1024 instances retain a lower per-byte cost than the SHA3-512 baseline across the full S1–S8 range, despite providing larger digest sizes and higher collision security levels. The ARM long-message results are particularly strong, with Dragon-512 falling below 5 cpb from S5 onward.

Summary. Overall, the measurements indicate that Dragon is efficient in software on both x86-64 and ARM. Its per-byte cost drops quickly once the fixed per-call overhead is amortized, and the long-message cpb remains low for all three fixed-output instances. The direct SHA3-512 comparison highlights the software performance advantage of the full Dragon family under the same S1–S8 ngcc benchmark setting: Dragon-512, Dragon-768, and Dragon-1024 all remain faster than SHA3-512 on both evaluated platforms. The results show a clear and predictable scaling pattern across the family: Dragon-512 gives the best raw speed, while Dragon-768 and Dragon-1024 provide higher security levels with moderate additional software cost.

5.2 Hardware performance

To comprehensively evaluate the hardware performance of the hash function, we implement its 512-, 768-, and 1024-bit versions in an ASIC setting. The hardware performance is illustrated in Table 16.

Following the algorithm specification, we implement the complete hash functions for the 512-, 768-, and 1024-bit versions. For each version, the underlying permutation adopts a round-based architecture in which one round is executed per clock cycle and the round logic is iteratively reused. The hardware designs are described in VHDL and functionally verified using ModelSim. The verified designs are then synthesized using Synopsys Design Compiler with a commercial UMC 55nm standard-cell library. During synthesis, the timing constraint is progressively tightened to achieve the maximum throughput. Table 16 reports the resulting area, critical-path delay, frequency, and power, together with the cycle count, throughput, and throughput-to-area ratio for message lengths of 32, 1024, and 65536 bytes, representing short, medium-sized, and long messages, respectively.

Table 16: Hardware performance of SHA3-512, Dragon-512, Dragon-768, and Dragon-1024 for message lengths of 32, 1024, and 65536 bytes. These implementations are synthesized at high frequencies to maximize throughput.

Algorithm	Implementation				32 B			1024 B			65536 B		
	Area (kGE)	Delay (ns)	Freq. (MHz)	Power (mW)	Cycles	TP (Mbps)	TP/Area (Mbps/kGE)	Cycles	TP (Mbps)	TP/Area (Mbps/kGE)	Cycles	TP (Mbps)	TP/Area (Mbps/kGE)
SHA3-512	44.68	0.84	1190.48	31.25	38	8020	179.48	512	19048	426.27	30976	20150	450.93
Dragon-512	122.28	3.93	254.45	13.80	30	2171	17.76	297	7018	57.40	17433	7653	62.58
Dragon-768	132.41	3.88	257.73	15.55	34	1941	14.66	338	6247	47.18	20498	6592	49.79
Dragon-1024	131.24	3.98	251.26	16.09	38	1693	12.90	449	4584	34.93	26657	4942	37.65

6 Discussion and Feature Statement

Comparison with SM3. SM3 [Sta10] is the current Chinese commercial hash standard and has received extensive public analysis. It produces a 256-bit digest, providing 128-bit collision and 256-bit (second) preimage security. Dragon is designed for the next-generation setting, in which the call for new commercial algorithms [NIC25] targets (second) preimage security up to 1024 bits, a range covered by its 512-, 768-, and 1024-bit members. In Section 5, Dragon-512 and Dragon-768 are faster than SM3 at every tested input length on both x86-64 and ARM, while Dragon-1024 attains 1024-bit (second) preimage security at a long-message cost close to that of SM3.

Dragon is a hash family built on the sponge-forward XOF mode Sponge-FP, pd10^* over a 1600-bit permutation, instantiated as Dragon-512, Dragon-768, Dragon-1024, and the extendable-output instance Dragon-XOF. Its main features are the following.

Dragon is innovative in both its mode and its permutation. Its sponge-forward mode, to appear at CRYPTO 2026, attains high (second) preimage security with a smaller permutation than a standard sponge requires. This is what allows a 1600-bit permutation

to meet a 1024-bit (second) preimage target. The underlying **Dragon- p** is a new 1600-bit ARX permutation.

Dragon is simple. The whole permutation is built from a single **UPDATE** transformation. Each round consists of a diagonal step and a column step, and each step applies **UPDATE** five times, to the five cyclic diagonals and then to the five columns. No instance-specific component is used, and all members share the same permutation and the same mode.

Dragon is flexible. From one design it provides digest lengths of 512, 768, and 1024 bits together with three extendable-output instances, covering the mandatory 512- and 1024-bit lengths and the optional 768-bit length. Its optimized implementation targets 64-bit platforms, and the design can also be implemented on 32-bit platforms. Crucially, **Dragon- p** is built exclusively from modular addition, rotation, and XOR, with no S-boxes and no table lookups. It therefore depends on no dedicated cryptographic instruction-set extension whatsoever, neither Intel’s AES-NI or SHA Extensions on x86-64, nor the Armv8 Cryptographic Extension (including the Armv8.2 AES, SHA-512, and SHA-3 instructions) on AArch64. Such extensions accelerate AES- and SHA-based primitives but are confined to specific vendors and microarchitectures, and the same code loses much of its speed on any processor that lacks them. **Dragon** has no such dependency: the very same implementation runs at full, predictable speed on every general-purpose processor and instruction set, from high-end x86-64 and ARM cores down to embedded and legacy targets that offer no cryptographic acceleration at all.

Dragon performs well across platforms. In software, all three fixed-output instances are faster than SHA3-512 at every tested message length on both x86-64 and ARM, as reported in Section 5. We also provide ASIC hardware implementations of the 512-, 768-, and 1024-bit instances.

A Differential and Linear Trails

This appendix lists the concrete trails reported in Section 4.2. A state is a 5×5 array of 64-bit lanes. We write the lane coordinates as (x, y) with $x, y \in \{0, \dots, 4\}$. In the tables below, columns are ordered from $x = 0$ to $x = 4$, and rows are ordered from $y = 4$ down to $y = 0$, i.e., from left to right and from bottom to top. For an r -round trail, only the round-input states are shown: $\Delta S^{(0)}, \dots, \Delta S^{(r)}$ for differential trails and $\Gamma S^{(0)}, \dots, \Gamma S^{(r)}$ for linear trails.

Table 17: One-round XOR-differential trail with probability 2^{-2} .

$\Delta S^{(0)}$					
$y \backslash x$	0	1	2	3	4
4	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x8000000000000000
3	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000
2	0x0000000000000000	0x0000000000000000	0x000000C000000000	0x0000000000000000	0x0000000000000000
1	0x0000000000000000	0x8000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000
0	0x4000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000

$\Delta S^{(1)}$					
$y \backslash x$	0	1	2	3	4
4	0x0001000000000001	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000
3	0x8000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000
2	0x0000000000000001	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000
1	0x8000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000
0	0x8000000000000001	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000

Table 18: Two-round XOR-differential trail with probability 2^{-45} .

$\Delta S^{(0)}$					
$y \backslash x$	0	1	2	3	4
4	0x0000000000000000	0x4000000000000000	0x000000000104020	0x0000000000000000	0x0020004020000000
3	0x0000000030000000	0x0000020800000000	0x0000000000000000	0x00000000000000C0	0x0000000040100000
2	0x0000200000000000	0x0000000000000000	0x0000000000004030	0x0000000000000000	0x0000003000000000
1	0x0000000000000000	0x0020084020000000	0x0020000000000000	0xC000000000000000	0x000000000104410
0	0x0000000010000000	0x8000000000000000	0xF000000000000000	0x0000000000000000	0x0000000000000000

$\Delta S^{(1)}$					
$y \backslash x$	0	1	2	3	4
4	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000
3	0x0000000040000000	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000
2	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000
1	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000
0	0x0000000000000000	0x0000000000000000	0x8000000000000000	0x0000000000000000	0x0000000000000000

$\Delta S^{(2)}$					
$y \backslash x$	0	1	2	3	4
4	0x0000000100020000	0x0000000000000201	0x0001000000000001	0x0000000000000000	0x0000000000000100
3	0x0000000000000000	0x80000000000000200	0x80000000000000000	0x0000000000000000	0x000000000000800000
2	0x0000000100000000	0x00000000000000201	0x0000000000000001	0x0000000000000000	0x0000000000000000
1	0x0000000100000000	0x00000000000000200	0x8000000000000000	0x0000000000000000	0x00000000000080000
0	0x0000000020000000	0x80000000000000201	0x80000000000000001	0x0000000000000000	0x00000000000080000

Table 19: One-round linear trail with absolute correlation 2^{-1} .

$\Gamma S^{(0)}$					
$y \backslash x$	0	1	2	3	4
4	0x0000000000000000	0x0000000000000001	0x0000000000000000	0x0000000000000000	0x0000000000000000
3	0x0000000080000000	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000
2	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000010000000000
1	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000001	0x0000000000000000
0	0x0000000000000000	0x0000000000000000	0x0000000000000001	0x0000000000000000	0x0000000000000000

$\Gamma S^{(1)}$					
$y \backslash x$	0	1	2	3	4
4	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000
3	0x0000000000000000	0x0000000000000000	0x0000000000000003	0x0000000000000000	0x0000000002000000
2	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000001800000
1	0x0000000000000000	0x0000000000000000	0x0000000000000001	0x0000000000000000	0x0000000000000000
0	0x0000000000000000	0x0000000000000000	0x0000000000000002	0x0000000000000000	0x0000000003000000

Table 20: Two-round linear trail with absolute correlation 2^{-24} .

$\Gamma S^{(0)}$					
$y \backslash x$	0	1	2	3	4
4	0x0000000000000000	0x0000000000000000	0x0000000030000000	0x0000000000000000	0x0000000000000000
3	0x0000000000000000	0x0000000000000060	0x0000000000000000	0x0000000000000000	0x0000000000000000
2	0x0000000000000400	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000
1	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000008020000000
0	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000

$\Gamma S^{(1)}$					
$y \backslash x$	0	1	2	3	4
4	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000
3	0x8000000000000001	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000
2	0x8000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000
1	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000
0	0x0000000000000001	0x0000000000000000	0x0000000000000000	0x0000000000000000	0x0000000000000000

$\Gamma S^{(2)}$					
$y \backslash x$	0	1	2	3	4
4	0x2000000000000000	0x0812000000003000	0x0000000000000000	0x0000004030060000	0x0000000000000000
3	0x0001800000000000	0x0300000000000000	0x0000000000000000	0x0000000402000000	0x0030000000000000
2	0x2000800000000000	0x0812000010003001	0x0000000000000000	0x0000084230060000	0x0010000000000000
1	0x0000000000001004	0x0200000000004008	0x0000000000000000	0x0020100401000000	0x0000000000000000
0	0x0001000000000004	0x0000000010000001	0x0000000000000000	0x0000080203000000	0x0020000000000000

References

- [AHMN10] Jean-Philippe Aumasson, Luca Henzen, Willi Meier, and María Naya-Plasencia. Quark: A lightweight hash. In Stefan Mangard and François-Xavier Standaert, editors, *CHES 2010*, volume 6225 of *LNCS*, pages 1–15, August 2010.
- [AMP11] Elena Andreeva, Bart Mennink, and Bart Preneel. Security reductions of the second round SHA-3 candidates. In Mike Burmester, Gene Tsudik, Spyros S. Magliveras, and Ivana Ilic, editors, *ISC 2010*, volume 6531 of *LNCS*, pages 39–53, October 2011.
- [AMP12] Elena Andreeva, Bart Mennink, and Bart Preneel. The parazoa family: generalizing the sponge hash functions. *Int. J. Inf. Sec.*, 11(3):149–165, 2012.
- [AS11] Elena Andreeva and Martijn Stam. The symbiosis between collision and preimage resistance. In Liqun Chen, editor, *13th IMA International Conference on Cryptography and Coding*, volume 7089 of *LNCS*, pages 152–171, December 2011.
- [BDKW19] Achiya Bar-On, Orr Dunkelman, Nathan Keller, and Ariel Weizman. DLCT: A new tool for differential-linear cryptanalysis. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part I*, volume 11476 of *LNCS*, pages 313–342, May 2019.
- [BDPV08] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. On the indifferentiability of the Sponge construction. In Nigel P. Smart, editor, *EUROCRYPT 2008*, volume 4965 of *LNCS*, pages 181–197, April 2008.
- [BDPVA07] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Sponge Functions. In *ECRYPT hash workshop*, volume 2007, 2007.
- [BDPVA11] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Cryptographic Sponge Functions. *2011-STMicroelectronics NXP Semiconductors, Version 0.1 January 14*, 2011.
- [Bey21] Tim Beyne. A geometric approach to linear cryptanalysis. *LNCS*, pages 36–66, 2021.
- [BHT98] Gilles Brassard, Peter Høyer, and Alain Tapp. Quantum cryptanalysis of hash and claw-free functions. In Claudio L. Lucchesi and Arnaldo V. Moura, editors, *LATIN 1998*, volume 1380 of *LNCS*, pages 163–169, April 1998.
- [BKL⁺11] Andrey Bogdanov, Miroslav Knežević, Gregor Leander, Deniz Toz, Kerem Varici, and Ingrid Verbauwhede. Spongint: A lightweight hash function. In Bart Preneel and Tsuyoshi Takagi, editors, *CHES 2011*, volume 6917 of *LNCS*, pages 312–325, September / October 2011.
- [BLP⁺08] Andrey Bogdanov, Gregor Leander, Christof Paar, Axel Poschmann, Matthew J. B. Robshaw, and Yannick Seurin. Hash functions and RFID tags: Mind the gap. In Elisabeth Oswald and Pankaj Rohatgi, editors, *CHES 2008*, volume 5154 of *LNCS*, pages 283–299, August 2008.
- [BS05] An Braeken and Igor Semaev. The ANF of the composition of addition and multiplication mod 2^n with a Boolean function. In Henri Gilbert and Helena Handschuh, editors, *FSE 2005*, volume 3557 of *LNCS*, pages 112–125. Springer, Heidelberg, February 2005.

- [CAD⁺20] David A Cooper, Daniel C Apon, Quynh H Dang, Michael S Davidson, Morris J Dworkin, Carl A Miller, et al. Recommendation for Stateful Hash-Based Signature Schemes. *NIST Special Publication*, 800(208):800–208, 2020.
- [CDMP05] Jean-Sébastien Coron, Yevgeniy Dodis, Cécile Malinaud, and Prashant Puniya. Merkle-Damgård revisited: How to construct a hash function. In Victor Shoup, editor, *CRYPTO 2005*, volume 3621 of *LNCS*, pages 430–448, August 2005.
- [CNS17] André Chailloux, María Naya-Plasencia, and André Schrottenloher. An efficient quantum collision search algorithm and implications on symmetric cryptography. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part II*, volume 10625 of *LNCS*, pages 211–240, December 2017.
- [D⁺15] Morris J Dworkin et al. SHA-3 standard: Permutation-based hash and extendable-output functions. 2015.
- [DGHM13] Gregory Demay, Peter Gaži, Martin Hirt, and Ueli Maurer. Resource-restricted indistinguishability. In Thomas Johansson and Phong Q. Nguyen, editors, *EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 664–683, May 2013.
- [DGPW12] Alexandre Duc, Jian Guo, Thomas Peyrin, and Lei Wei. Unaligned rebound attack: Application to Keccak. In Anne Canteaut, editor, *FSE 2012*, volume 7549 of *LNCS*, pages 402–421. Springer, Heidelberg, March 2012.
- [Foe23] Robin Foekens. Security of the Sponge Construction with a Random Transformation. Bachelor’s thesis, Radboud University, Nijmegen, 2023. https://www.cs.ru.nl/bachelors-theses/2023/Robin_Foekens___4666615___Security_of_the_Sponge_Construction_with_a_Random_Transformation.pdf.
- [FWG⁺16] Kai Fu, Meiqin Wang, Yinghua Guo, Siwei Sun, and Lei Hu. MILP-based automatic search algorithms for differential and linear trails for speck. In Thomas Peyrin, editor, *FSE 2016*, volume 9783 of *LNCS*, pages 268–288. Springer, Heidelberg, March 2016.
- [GHJ⁺25] Chun Guo, Kai Hu, Shuntian Jiang, Yanhong Fan, Yong Fu, Bart Preneel, and Meiqin Wang. Permutation-Based Hash from Non-Idealized Assumptions: Adding Feed-Forward to Sponge. Cryptology ePrint Archive, Paper 2025/1006, 2025. To appear at CRYPTO 2026.
- [GPP11] Jian Guo, Thomas Peyrin, and Axel Poschmann. The PHOTON family of lightweight hash functions. In Phillip Rogaway, editor, *CRYPTO 2011*, volume 6841 of *LNCS*, pages 222–239, August 2011.
- [Gro96] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *28th ACM STOC*, pages 212–219. ACM Press, May 1996.
- [HNW26] Kai Hu, Zhongfeng Niu, and Meiqin Wang. Round-Based Approximation of (Higher-Order) Differential-Linear Correlation – A Geometric Approach Perspective. In *Advances in Cryptology – EUROCRYPT 2026*, volume 16004 of *Lecture Notes in Computer Science*. Springer, 2026.
- [HPY12] Shoichi Hirose, Je Hong Park, and Aaram Yun. A simple variant of the Merkle-Damgård scheme with a permutation. *Journal of Cryptology*, 25(2):271–309, April 2012.

- [HSWW20] Kai Hu, Siwei Sun, Meiqin Wang, and Qingju Wang. An algebraic formulation of the division property: Revisiting degree evaluations, cube attacks, and key-independent sums. In *ASIACRYPT 2020, Part I*, LNCS, pages 446–476, December 2020.
- [HY24] Kai Hu and Trevor Yap. Perfect monomial prediction for modular addition. *IACR Trans. Symm. Cryptol.*, 2024(3):177–199, 2024.
- [Jou04] Antoine Joux. Multicollisions in iterated hash functions. Application to cascaded constructions. In Matthew Franklin, editor, *CRYPTO 2004*, volume 3152 of LNCS, pages 306–316, August 2004.
- [KK06] John Kelsey and Tadayoshi Kohno. Herding hash functions and the Nistradamus attack. In Serge Vaudenay, editor, *EUROCRYPT 2006*, volume 4004 of LNCS, pages 183–200, May / June 2006.
- [KS05] John Kelsey and Bruce Schneier. Second preimages on n-bit hash functions for much less than 2^n work. In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of LNCS, pages 474–490, May 2005.
- [Lef23] Charlotte Lefevre. Indifferentiability of the Sponge construction with a restricted number of message blocks. *IACR Trans. Symm. Cryptol.*, 2023(1):224–243, 2023.
- [LH94] Susan K. Langford and Martin E. Hellman. Differential-linear cryptanalysis. In Yvo Desmedt, editor, *CRYPTO’94*, volume 839 of LNCS, pages 17–25, August 1994.
- [LM95] Frank T Leighton and Silvio Micali. Large provably fast and secure digital signature schemes based on secure hash functions, July 11 1995. US Patent 5,432,852.
- [LM02] Helger Lipmaa and Shiho Moriai. Efficient algorithms for computing differential properties of addition. In Mitsuru Matsui, editor, *FSE 2001*, volume 2355 of LNCS, pages 336–350. Springer, Heidelberg, April 2002.
- [LM22] Charlotte Lefevre and Bart Mennink. Tight preimage resistance of the Sponge construction. LNCS, pages 185–204, 2022.
- [Mer79] Ralph Charles Merkle. *Secrecy, authentication, and public key systems*. Stanford university, 1979.
- [MRST09] Florian Mendel, Christian Rechberger, Martin Schl  ffer, and S  ren S. Thomsen. The rebound attack: Cryptanalysis of reduced Whirlpool and Gr  stl. In Orr Dunkelman, editor, *FSE 2009*, volume 5665 of LNCS, pages 260–276. Springer, Heidelberg, February 2009.
- [MVDP11] Nicky Mouha, Vesselin Velichkov, Christophe De Canni  re, and Bart Preneel. The differential analysis of S-functions. In Alex Biryukov, Guang Gong, and Douglas R. Stinson, editors, *SAC 2010*, volume 6544 of LNCS, pages 36–56, August 2011.
- [Nat15] National Institute of Standards and Technology. Secure Hash Standard (SHS). *FIPS PUB*, 180(4), 2015.
- [NBHW26] Zhongfeng Niu, Tim Beyne, Kai Hu, and Meiqin Wang. Cryptanalytic properties of Mealy machines. In *Advances in Cryptology – CRYPTO 2026*. Springer, 2026. To appear. <https://eprint.iacr.org/2026/1193>.

- [NHS⁺24] Zhongfeng Niu, Kai Hu, Siwei Sun, Zhiyu Zhang, and Meiqin Wang. Speeding up preimage and key-recovery attacks with highly biased differential-linear approximations. *LNCS*, pages 73–104, 2024.
- [NIC25] NICCS. Announcement on launching the next-generation commercial cryptographic algorithms program, 2025. https://www.niccs.org.cn/niccs/Notice/pc/content/content_1937427069632253952.html.
- [RS04] Phillip Rogaway and Thomas Shrimpton. Cryptographic hash-function basics: Definitions, implications, and separations for preimage resistance, second-preimage resistance, and collision resistance. In Bimal K. Roy and Willi Meier, editors, *FSE 2004*, volume 3017 of *LNCS*, pages 371–388. Springer, Heidelberg, February 2004.
- [RSS11] Thomas Ristenpart, Hovav Shacham, and Thomas Shrimpton. Careful with composition: Limitations of the indistinguishability framework. In Kenneth G. Paterson, editor, *EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 487–506, May 2011.
- [SLZ⁺25] Siwei Sun, Shun Li, Zhiyu Zhang, Charlotte Lefevre, Bart Mennink, Zhen Qin, and Dengguo Feng. Permutation-based hashing with stronger (second) preimage resistance. *Cryptology ePrint Archive*, Report 2025/963, 2025.
- [Sta10] State Cryptography Administration of China. SM3 cryptographic hash algorithm. <https://www.oscca.gov.cn/sca/xxgk/2010-12/17/1002389/files/302a3ada057c4a73830536d03e683110.pdf>, December 2010.
- [STMC⁺24] Meltem Sönmez Turan, Kerry McKay, Donghoon Chang, Jinkeon Kang, and John Kelsey. Ascon-Based Lightweight Cryptography Standards for Constrained Devices: Authenticated Encryption, Hash, and Extendable Output Functions. Technical report, National Institute of Standards and Technology, 2024.
- [Wal03] Johan Wallén. Linear approximations of addition modulo 2^n . In Thomas Johansson, editor, *FSE 2003*, volume 2887 of *LNCS*, pages 261–273. Springer, Heidelberg, February 2003.
- [Yas08] Kan Yasuda. How to fill up Merkle-Damgård hash functions. In Josef Pieprzyk, editor, *ASIACRYPT 2008*, volume 5350 of *LNCS*, pages 272–289, December 2008.